A* and Heuristics

Contents

1. Overview . . . 3
2. A guess of the distance left: the heuristic . . . 5
3. Following the guess alone: greedy best-first search . . . 8
4. A*: cost so far plus the guess . . . 10
   - 4.1 g, h and f on the first step . . . 10
   - 4.2 The smallest f first, ties to the smaller h . . . 13
   - 4.3 A shorter way in lowers g . . . 13
   - 4.4 Parent arrows give the route . . . 14
   - 4.5 The algorithm . . . 14
5. Why A* returns the shortest route: admissible heuristics . . . 14
   - 5.1 A guess that never overestimates . . . 15
   - 5.2 Why the first route out is the shortest . . . 16
   - 5.3 A closer guess takes fewer cells . . . 17
6. When admissible is not enough: consistent heuristics . . . 18
   - 6.1 The closed list throws away a shorter way in . . . 18
   - 6.2 The fix: a guess that never drops faster than the distance . . . 20
   - 6.3 Why consistency keeps g right . . . 22
7. Saving memory: iterative deepening and IDA* . . . 23
8. Searching from the door: backward and bidirectional search . . . 26
   - 8.1 Backward search . . . 26
   - 8.2 Bidirectional search . . . 28
   - 8.3 When to stop, and where the route is . . . 28
9. Several goals: multi-goal A* . . . 30
   - 9.1 The virtual goal . . . 31
   - 9.2 A moving goal . . . 34
10. Trading length for time: weighted A* and anytime A* . . . 37
    - 10.1 Weighted A* . . . 37
    - 10.2 Anytime A* and ARA* . . . 39
11. Skipping equal routes: jump point search . . . 40
12. Routes at any angle: Theta* . . . 42
13. Summary . . . 45
14. Sources . . . 46

15. Key terms 48

Where this fits:

- **Builds on:** Vector magnitude, distance and scalar operations; Depth-first search; Dijkstra's algorithm.
- **Leads to:** Forward state-space search.
- **Compare with:** Breadth-first search; Dijkstra's algorithm.

1. Overview

Key point: Dijkstra's algorithm finds the robot's shortest route to the door, but it searches away from the door as much as towards it. Adding a guess of the distance left that never overestimates makes the search head for the door: A* finds the same 3.414 m route after 9 cells instead of 54.

The robot at (2 m, 1 m) can already find a route to the door on its map of the room, cut into 0.5 m cells, 76 of them free (occupancy grid (G-2340)). The robot moves from a free cell to any of its 8 neighbours: 0.5 m straight or 0.707 m diagonally, and a diagonal move never cuts the corner of a blocked cell. The robot stands on the corner of the start cell S, which runs from 2.0 to 2.5 m along x and 1.0 to 1.5 m along y; the goal cell G sits just under the door, from 3.5 to 4.0 m along x and 3.5 to 4.0 m along y.

On this grid, Dijkstra's algorithm finds the shortest route from S to G: 3.414 m. Dijkstra keeps an open list of the cells it has reached, the **frontier** (G-2547), and always takes off the cell nearest to S.

What still breaks is the work this costs. Figure 1 (top) plays Dijkstra's search one cell per frame: watch the red cells, the ones taken off the open list, grow as a ring around S, down to the bottom wall and left to the pole, where no route to the door goes.

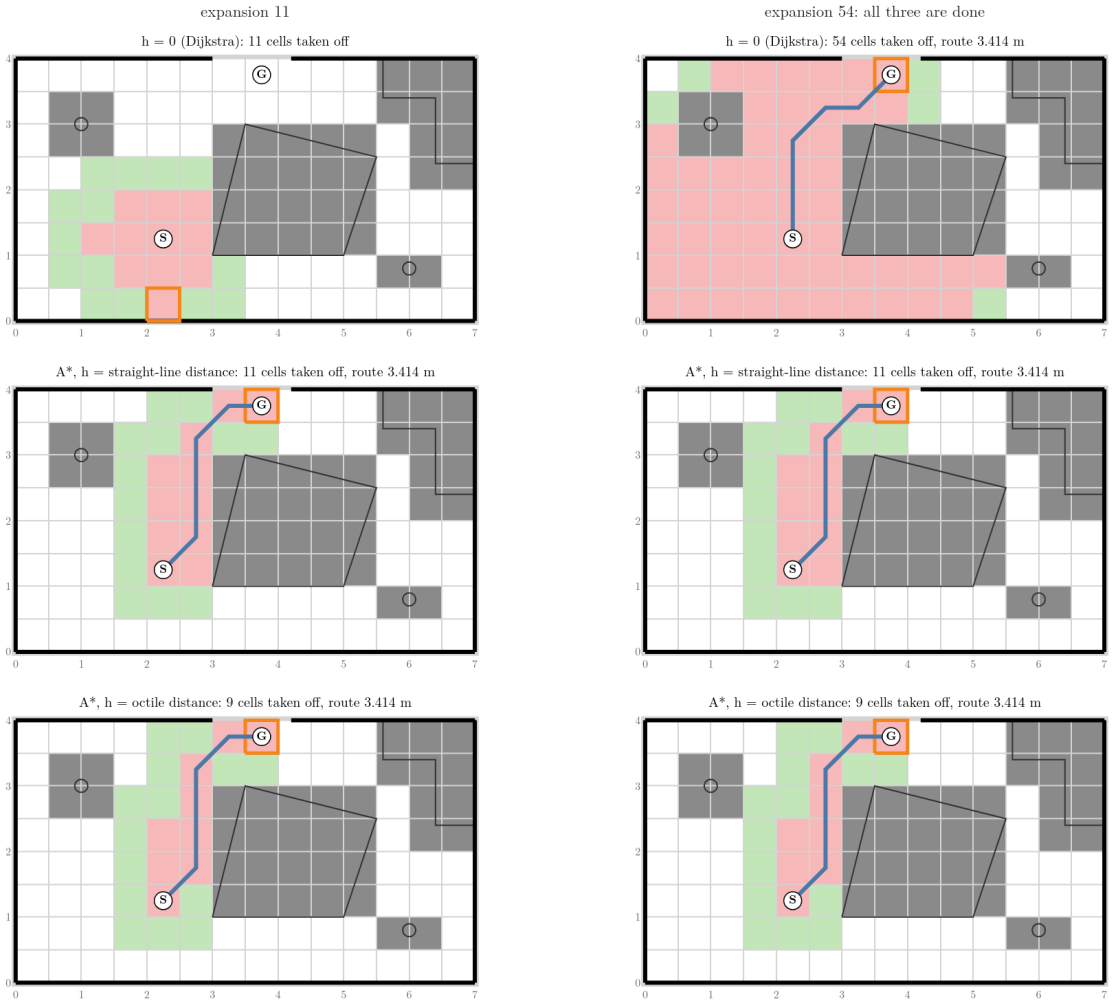


Figure 1: The same search loop with three guesses h of the distance left, on the room grid from the start cell S to the goal cell G under the door, one cell taken off the open list per frame in all three panels (two per frame after frame "expansion 11", when only Dijkstra still changes). Top: h = 0, which is Dijkstra's algorithm. Middle: A* with the straight-line distance to G. Bottom: A* with the octile distance to G. Light red: cells taken off the open list; light green: cells on it; orange outline: the cell just taken off; grey: blocked cells. A* with the octile distance is done after 9 cells, with the straight line after 11, and each holds its last frame; Dijkstra spreads in every direction and is done after 54. All three routes (blue) are 3.414 m long; Dijkstra's differs in shape because several routes tie at that length

Dijkstra takes 54 cells off its open list in all: every free cell nearer to S than 3.414 m, 53 of them, and then G (notebook). Dijkstra ranks cells only by the distance already driven, so it knows nothing about where the door is and spreads evenly in every direction (CS188 §1.4).

The other two panels of Figure 1 run the search this Note builds, A*: the same loop, with a guess of the distance left added to each cell's score. With the straight-line distance as the guess (middle) it reaches G after 11 cells, with the octile distance of Section 2 (bottom) after

9, and both return the same 3.414 m. This Note adds the one thing Dijkstra lacks, a guess of the distance left, and makes it safe to use:

- the guess itself, the heuristic (Section 2);
- searching by the guess alone, which is fast but can go the wrong way (Section 3);
- A*, which adds the guess to the distance driven (Section 4);
- why A* still returns the shortest route when the guess never overestimates (Section 5), and the stronger rule it needs when it never revisits a cell (Section 6);
- variants for less memory (Section 7), for searching from the door (Section 8), for several goals (Section 9), for less time (Section 10), for fewer cells on open grids (Section 11), and for routes that are not limited to 45° turns (Section 12).

2. A guess of the distance left: the heuristic

> **Key point:** Dijkstra cannot tell a cell near the door from a cell far from it. A cheap guess of the distance left from each cell, worked out as if there were no obstacles, gives it that sense of direction: from S the guess is 3.121 m, the true distance 3.414 m.

The idea. Dijkstra ranks the cells on its open list by the distance driven from S, so a cell 1 m behind the robot ranks with a cell 1 m towards the door. To prefer the second, the search needs, for each cell, some idea of how far the door still is. The idea has to be cheap: the exact distance left is what we are searching for.

A traveller does this with a map: before knowing the roads, the distance as the crow flies already tells which town is nearer. The robot can do the same on its grid: count the moves to G as if the table, the cabinet and the poles were not there.

The standard term. A function that guesses the cost still to go from a cell to the goal is a **heuristic** (G-2570), written h (CS188 §1.4; LaValle §2.2.2). Figure 2 writes the guess for our room in every free cell.

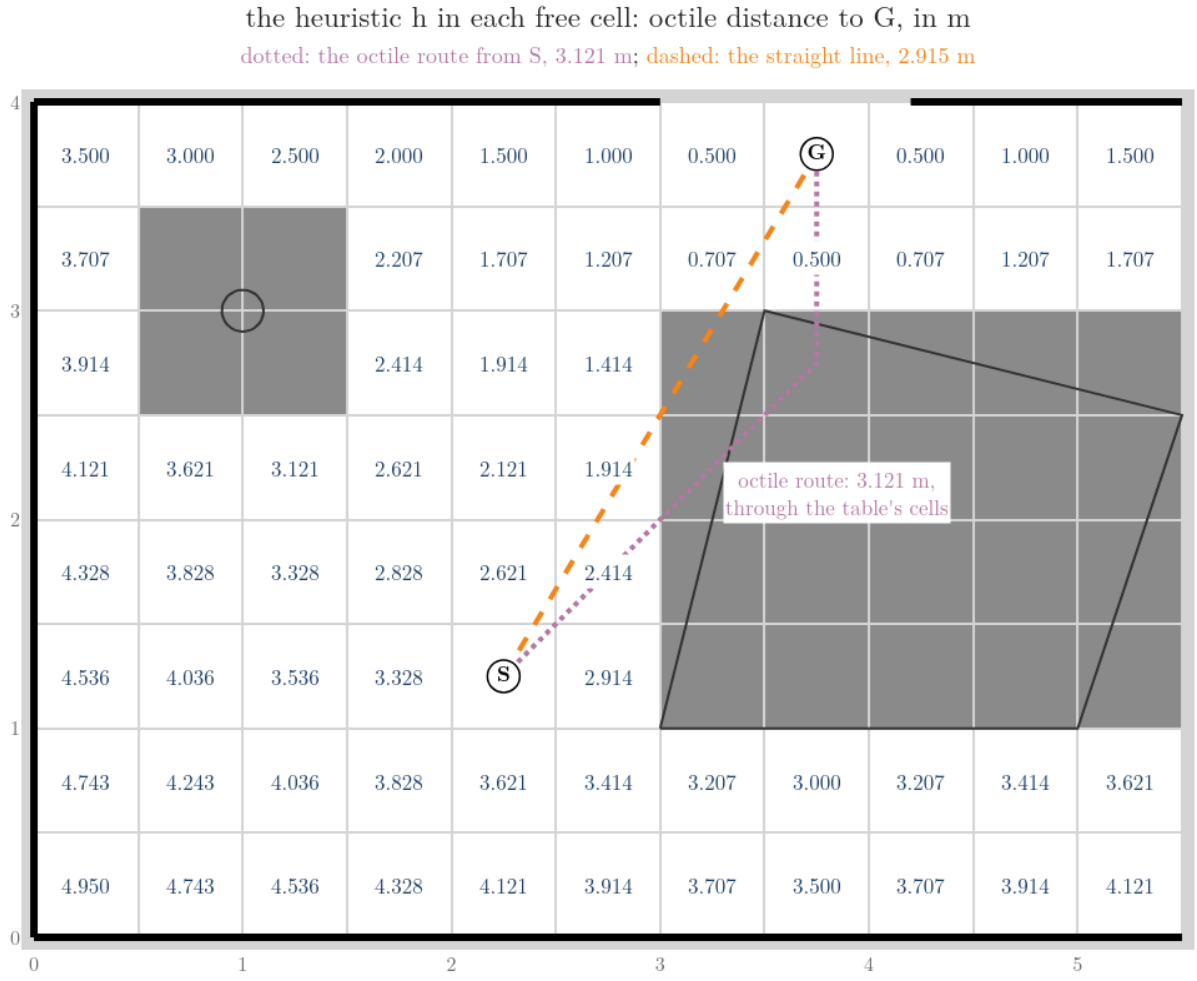


Figure 2: The heuristic h in every free cell of the left part of the room (x 0 to 5.5 m): the octile distance to G, in m, the length of the shortest 8-connected route if there were no obstacles. From S it is 3.121 m, the dotted route of 3 diagonal and 2 straight moves, which runs through the table's cells; the dashed line is the straight-line distance, 2.915 m. The true shortest route from S is 3.414 m

The mechanism on our numbers. On an empty 8-connected grid the shortest route makes diagonal moves while both the column count and the row count still have to shrink, and straight moves for what is left (Patel, Heuristics). S is 3 columns left of G and 5 rows below it, so with:

- Δx, the number of columns between the cell and G, 3 for S;
- Δy, the number of rows between the cell and G, 5 for S;
- m, the smaller of Δx and Δy, 3 for S;
- M, the larger of them, 5 for S;
- a, the length of a straight move, 0.5 m;
- ℓ, the length of a diagonal move, 0.707 m.

The route takes m diagonal moves, which cover the 3 columns and 3 of the rows, and then $M - m$ straight moves for the 2 rows left:

$$h(S) = 3 \times 0.707 + 2 \times 0.5$$

$$= 2.121 + 1.0$$

$$= 3.121 \text{ m}$$

The same in symbols:

$$h = \ell \times m + a \times (M - m)$$

This heuristic is the **octile distance** (G-2571) (Patel, Heuristics). The octile distance is the exact answer to an easier problem, the same grid with every obstacle removed; dropping a rule of the real problem in this way is a common source of heuristics (CS188 §1.4).

A second heuristic is the straight line from the centre of S to the centre of G, the **Euclidean distance** (G-715), which drops one more rule: that moves follow the grid. With 3 columns and 5 rows of 0.5 m:

$$\text{straight line} = 0.5 \times \sqrt{3^2 + 5^2}$$

$$= 0.5 \times \sqrt{34}$$

$$= 0.5 \times 5.83095$$

$$= 2.915 \text{ m}$$

Both heuristics cost a few multiplications per cell, and both fall short of the true 3.414 m, because the table makes the real route bend. The heuristic is ready; the next question is how the search should use it.

3. Following the guess alone: greedy best-first search

> **Key point:** Taking off the open list the cell whose heuristic h, the guess of the distance left, is smallest heads straight for the goal and looks at few cells, but it never counts the distance already driven, so it can return a long route: 9.536 m where 8.000 m exists.

The idea. The simplest use of the heuristic from Section 2: keep the same open list, but always take off the cell that looks nearest to the goal, the one with the smallest h. On the door trip this works: it reaches G after 7 cells with the 3.414 m route (notebook).

The standard term. Expanding, at every step, the open cell with the smallest heuristic value is **greedy best-first search** (G-2573) (CS188 §1.4). "Best-first" names any search that takes off the open list the cell that scores best; "greedy" says the score is the heuristic alone.

Where it goes wrong. The door trip is too short to show the flaw, so we give the robot a longer trip in the same room: from the bottom-left corner cell (x 0 to 0.5 m, y 0 to 0.5 m) to a cell at the right wall (x 6.5 to 7.0 m, y 1.5 to 2.0 m), behind the table. Figure 3 runs greedy best-first search on it: watch the red cells head up and to the right, straight at the goal, until the table's left side stops them, and then slide down the table to the bottom of the room.

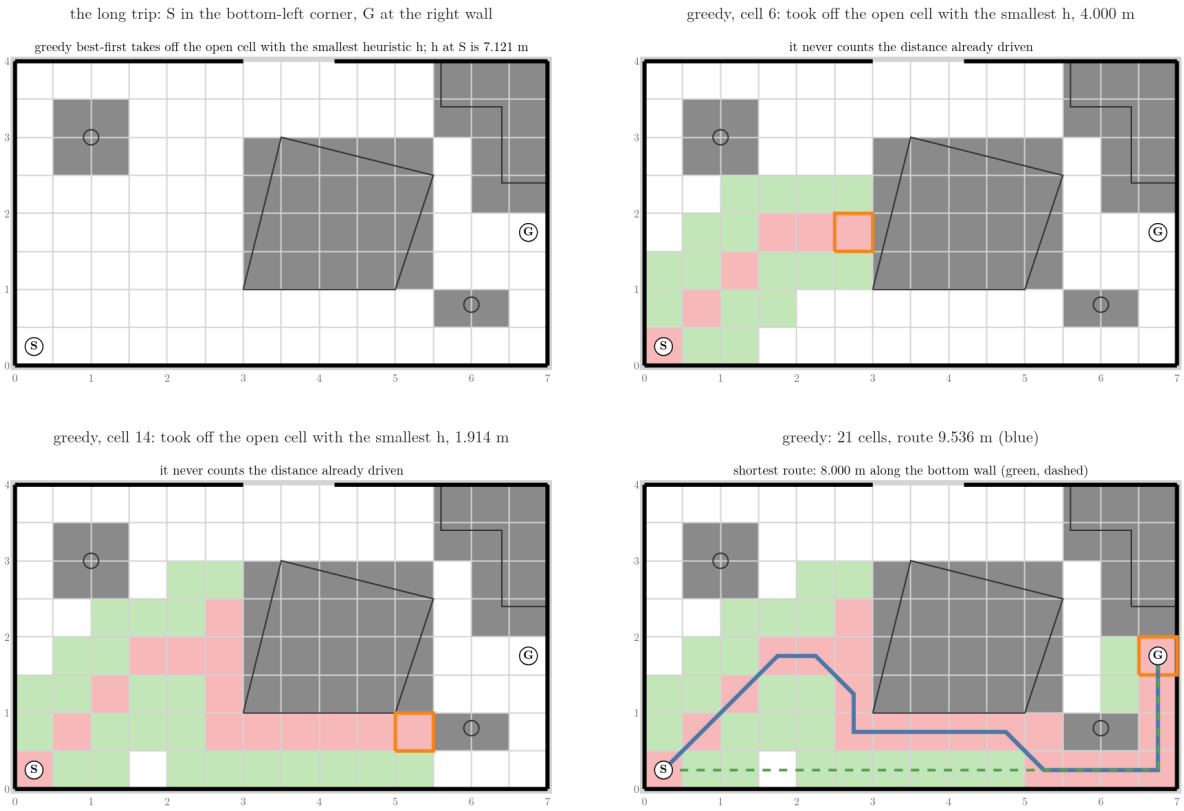


Figure 3: Greedy best-first search on the long trip, from S in the bottom-left corner cell to G at the right wall, one cell per frame: it always takes off the open list the cell with the smallest heuristic h, the octile distance to G (7.121 m at S), and never looks at the distance already driven. The search heads up and right, meets the table, and slides down its left side. Last frame: its route, 9.536 m after 21 cells (blue), and the shortest route, 8.000 m along the bottom wall (green, dashed)

Greedy best-first search returns 9.536 m after 21 cells; the shortest route, along the bottom wall, is 8.000 m (notebook). The climb towards the goal's height looked good to the heuristic, and nothing in the score counted what the climb and the way back down cost:

$$\text{extra} = 9.536 - 8.000$$

$$= 1.536 \text{ m}$$

Greedy search can go straight to the goal or wander like a badly guided depth-first search, depending on the obstacles (CS188 §1.4). The fix is to score each cell by both what it has cost and what it still promises.

4. A*: cost so far plus the guess

> **Key point:** A* scores every cell on the open list by $f = g + h$: the length of the best route found so far from S to the cell, plus the heuristic, the guess of the length left. A* takes off the cell with the smallest f, so it heads for the door like greedy search but drops a route as soon as its distance driven makes it look worse than another.

Greedy search of Section 3 forgot the distance driven; Dijkstra forgot the distance left. A* adds the two. We walk it through on the door trip, where every step can be checked by hand (CMU 16-350, A*; Hart et al. 1968).

4.1 g, h and f on the first step

For every cell the search has reached, it keeps three numbers:

- g, the length of the best route found so far from S to the cell, in m: the **cost so far** (G-2574), also called the g-value;
- h, the heuristic: the octile distance, a guess of the length left from the cell to G, in m (Section 2);
- f, their sum, in m: the **estimated total cost** (G-2575), also called the f-value, the length of the best route through the cell if the heuristic were exact.

$$f = g + h$$

At the start only S is on the open list, with $g = 0$ and $h = 3.121$ m, so $f = 3.121$ m. A* takes S off and expands it, as in breadth-first search (G-2550): it puts each of its 8 neighbours on the open list, here with its own g, h and f. For the cell diagonally up and to the right of S, 2 columns left of G and 4 rows below it:

$$g = 0.707$$

$$h = 2 \times 0.707 + 2 \times 0.5$$

$$= 2.414$$

$$f = 0.707 + 2.414$$

$$= 3.121 \text{ m}$$

For the cell straight above S, 3 columns and 4 rows from G:

$$g = 0.5$$

$$h = 3 \times 0.707 + 1 \times 0.5$$

$$= 2.621$$

$$f = 0.5 + 2.621$$

$$= 3.121 \text{ m}$$

Figure 4, frame “1. take S off”, shows all 8 neighbours with their numbers in the cell: g top left, h top right, f in the middle.

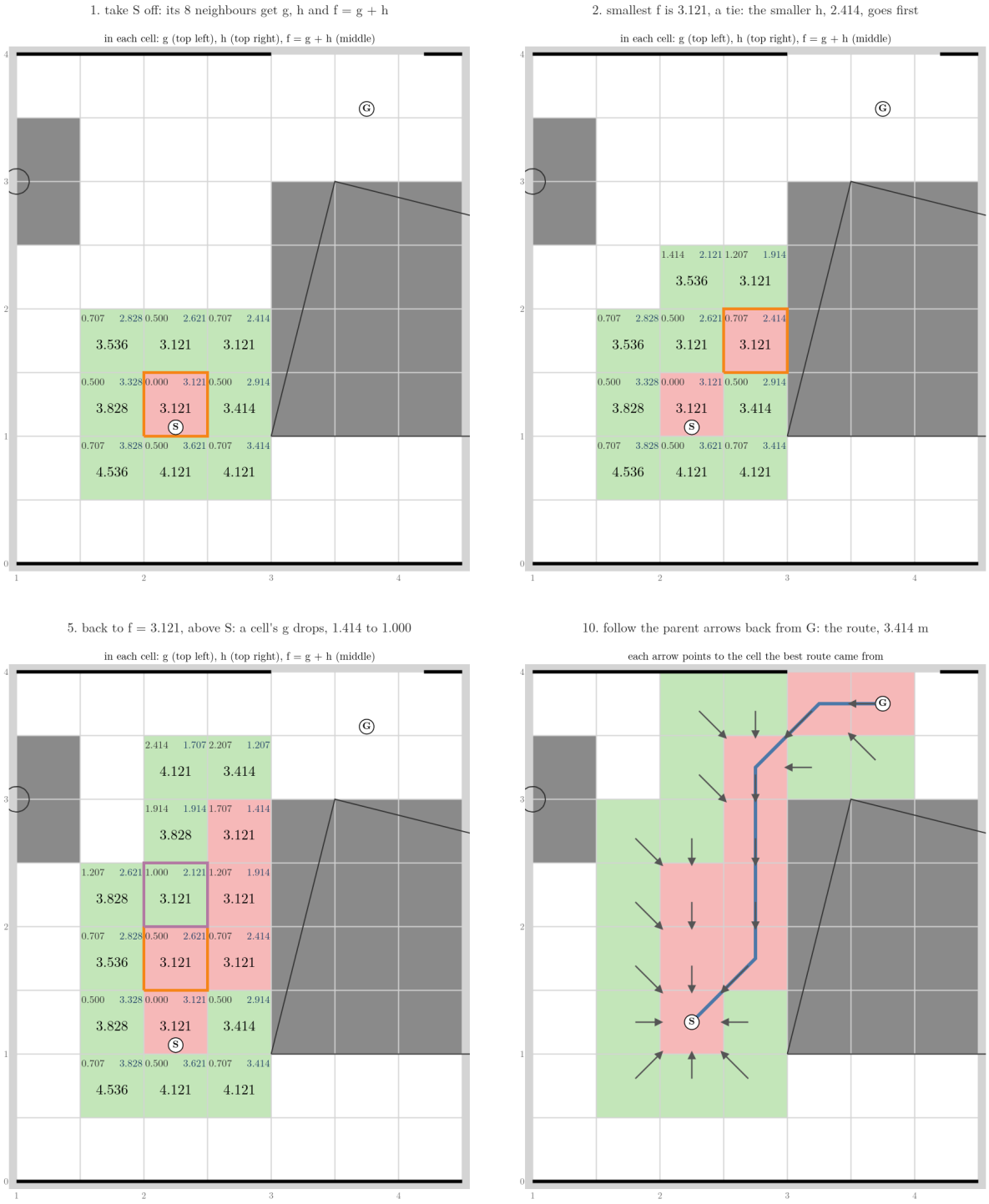


Figure 4: A* on the door trip step by step, on the part of the room it touches (x 1.0 to 4.5 m, y 0 to 4 m). Each open (green) or closed (red) cell shows g, the cost so far from S (top left), h, the heuristic, the octile distance to G (top right), and f = g + h (middle). One expansion per frame: the open cell with the smallest f comes off the open list (orange outline), ties going to the smaller h. In frames "5" and "6" a cell's g drops because a shorter way in has been found (purple outline): 1.414 to 1.000 and 1.914 to 1.500. In the last frame each arrow points to the cell the best route came from; following them back from G gives the route, 3.414 m

4.2 The smallest f first, ties to the smaller h

A* now takes off the open list the cell with the smallest f, as Dijkstra takes the smallest g. Two cells tie at 3.121 m, the cell up and to the right of S and the cell above S. A tie is broken by the smaller heuristic, which means the cell nearer to G by the heuristic (Patel, Heuristics): 2.414 m beats 2.621 m, so the cell up-right of S goes first (frame "2").

Frames "2" to "4" show why the smallest f is a good choice. While the search heads straight for G, each move adds to g exactly what it takes off h, so f stays at 3.121 m. Moving up from the cell up-right of S:

$$g = 0.707 + 0.5 = 1.207$$

$$h = 2.414 - 0.5 = 1.914$$

$$f = 1.207 + 1.914 = 3.121 \text{ m}$$

At the cell below the table's corner (frame "4") the diagonal towards G would cut the corner of a blocked table cell, which is not allowed. The cell above it gets $f = 3.414$ m: the detour round the corner costs what the heuristic did not see,

$$3.414 - 3.121 = 0.293 \text{ m}$$

4.3 A shorter way in lowers g

The cells straight up from the cell up-right of S are used up: the next one has $f = 3.414$ m. So A* goes back to the other cell with $f = 3.121$ m, the one straight above S (frame "5"). From there it reaches the cell two above S again. That cell already had $g = 1.414$ m from the first time it was reached, by two diagonal moves; the new way in is two straight moves:

$$g_{\text{old}} = 0.707 + 0.707 = 1.414 \text{ m}$$

$$g_{\text{new}} = 0.5 + 0.5 = 1.0 \text{ m}$$

The new way is shorter, so A* lowers the cell's g to 1.0 m and its f from 3.536 to 3.121 m (CMU 16-350, A*). Frame "6" does the same one cell higher: 1.914 m falls to 1.5 m. A cell on the open list keeps the best g found so far, never the first.

4.4 Parent arrows give the route

Now every f left is 3.414 m or more. Three cells tie at 3.414 m, and the smallest heuristic, 1.207 m, wins (frame "7"); then the cell left of G (frame "8"), then G itself (frame "9"). G comes off the open list with $h = 0$, so its f is its g: 3.414 m, the route length.

To find the route itself, each cell also stores the cell its best g came from: its parent (G-2576), the back pointer breadth-first search keeps too, here changed whenever g drops (CMU 16-350, A*). The last frame of Figure 4 draws the parents as arrows. Following them back from G to S gives the route: up-right from S, three cells up, past the table's corner, and right into G.

4.5 The algorithm

The steps of Sections 4.1 to 4.4, with the open list kept as a **priority queue** (G-2568) ordered by f and the closed list as the **visited list** (G-2548) of cells already expanded (CMU 16-350, A*; LaValle §2.2.2):

1. Put S on the open list with $g = 0$ and $f = h(S)$.
2. Take off the open list the cell with the smallest f (ties: the smallest h) and put it on the closed list.
3. If that cell is G, stop: follow the parents back to S.
4. Otherwise, for each neighbour that is free and not on the closed list, work out g through the current cell. If the neighbour is new, or this g is smaller than its stored g, store the new g, f and parent and put it on the open list.
5. Go to step 2.

The loop above is **A*** (G-2577), "A star" (Hart et al. 1968): Dijkstra's algorithm with the open list sorted by $f = g + h$ instead of g. With $h = 0$ everywhere, f is g and A* is Dijkstra (LaValle §2.2.2; Patel, Heuristics).

Figure 1 compared exactly these two, top and bottom: 54 cells with $h = 0$ against 9 with the octile heuristic, for the same 3.414 m. A* returned the shortest route here, but nothing so far says it always will.

5. Why A* returns the shortest route: admissible heuristics

> **Key point:** If the heuristic h is never more than the true distance left, f, the cost so far plus the heuristic, is a lower bound on any route through a cell, so the first route to G off the open list is the shortest. The heuristic is the one free choice in A*: among heuristics that never overestimate, a closer one takes fewer cells, 54, 11 and 9 for $h = 0$, the straight line and the octile distance, all with the 3.414 m route.

A* stopped at 3.414 m after only 9 of the 76 cells, without looking at most of the room. Section 5 checks why the cells it never expanded cannot hide a shorter route.

5.1 A guess that never overestimates

A trip by road is never shorter than the distance as the crow flies, so "crow-flight distance so far plus crow-flight distance left" can never overshoot the trip (Wikipedia, A* search algorithm). A* relies on the same property. A heuristic that is never more than the true cost left from any cell is an **admissible heuristic** (G-2578) (CS188 §1.4). With:

- $h(n)$, the heuristic at a cell n, in m (3.121 m at S);
- $h^*(n)$, the true shortest distance left from n to G, in m (3.414 m at S).

The rule is, for every cell:

$$h(n) \leq h^*(n)$$

The octile heuristic is admissible because it is the length of the shortest route in the room with its obstacles removed (Section 2), and putting obstacles back can only remove moves, never add a shorter one (LaValle §2.2.2). Figure 5 checks this on every free cell: the true distances come from one Dijkstra run backwards from G, and every dot lies on or below the line where guess and truth are equal.

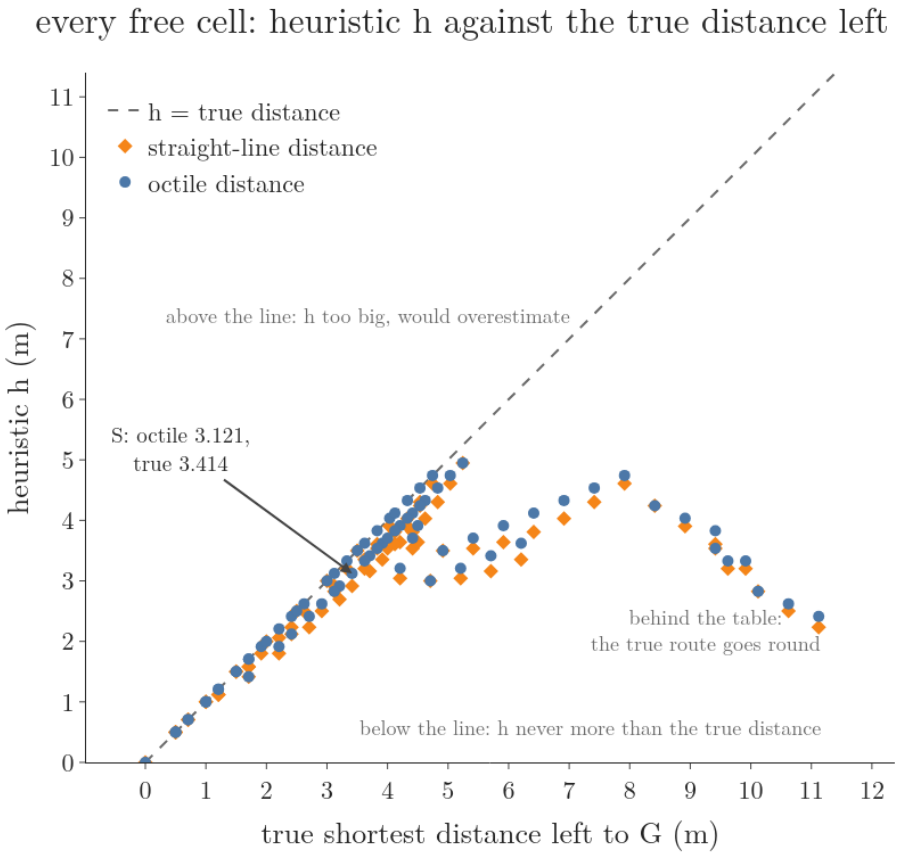


Figure 5: Every free cell, heuristic against the true distance left to G: across, the true shortest distance, from one Dijkstra run backwards from G; up, the heuristic h, the octile distance (blue dots) and the straight-line distance (orange diamonds). Every dot lies on or below the dashed line h = true distance, so neither guess overestimates; the octile dot of each cell is on or above its straight-line dot. S: octile 3.121 m, true 3.414 m. The cells far below the line lie behind the table, where the true route goes all the way round

5.2 Why the first route out is the shortest

When G comes off the open list, every other route from S to G is still unfinished. Any such route starts in a closed cell, S, and must leave the closed cells somewhere: the first cell it reaches outside them is on the open list. Take the cell left of S, which is on the open list when G comes off (Figure 4, frame "9"), with:

- n, that open cell;
- $g(n)$, its cost so far, 0.5 m, one straight move from S, already the shortest way to it;
- $h(n)$, its heuristic, 3.328 m;
- $f(n)$, its estimated total cost, 3.828 m;
- $L(n)$, the length of the shortest route from S to G through n, in m, unknown;
- $g(G)$, the length A* returns, 3.414 m; since $h(G) = 0$, it is also $f(G)$.

$$L(n) = 0.5 + h^*(n)$$

$$\geq 0.5 + 3.328$$

$$= 3.828$$

$$\geq 3.414$$

$$= g(G)$$

The same steps in symbols:

$$L(n) = g(n) + h^*(n)$$

$$\geq g(n) + h(n)$$

$$= f(n)$$

$$\geq f(G)$$

$$= g(G)$$

The second line is the admissible rule; the fourth holds because A* took G off before n, so $f(G)$ was the smallest f on the open list. Every open cell gives the same chain, so no route is shorter than the one A* returned (CS188 §1.4). The first line needs $g(n)$ to be the shortest way to n; it is here, and Section 6 shows when the closed list breaks it.

The chain also shows what a heuristic that overestimates does. If h could be larger than h^*, the second line would fail, $f(n)$ could exceed the length of the best route through n, and A* could take G off before ever expanding n: it might return a longer route, though faster (Patel, Heuristics). Section 10 uses that trade on purpose.

5.3 A closer guess takes fewer cells

The loop of Section 4.5 never changes; the heuristic h is the one part we choose, and it decides how much the loop searches. Any admissible heuristic keeps the answer right; the closer it is to the true distance left, the fewer cells A* takes off the open list. Figure 1 ran the same loop on the door trip with three admissible heuristics, one per panel (notebook):

Heuristic h	h at S (m)	Cells taken off the open list	Route (m)
0 everywhere (Dijkstra)	0	54	3.414
straight-line distance	2.915	11	3.414
octile distance	3.121	9	3.414

The octile distance is exact on an open 8-connected grid (Section 2); with the blocked cells it falls short wherever the true route must bend round an obstacle, and it stays admissible. In Figure 5 every octile dot is on or above the straight-line dot of the same cell, and 31 of the 76 octile dots lie on the line itself.

An admissible heuristic that is never smaller than another, cell by cell, **dominates** it (G-2579) (CS188 §1.4): the octile distance dominates the straight line, which dominates 0. A cell is taken off the open list only if its f is at most the route's length (Hart et al. 1968), and a larger admissible heuristic pushes more cells' f above it, so fewer cells look as promising as the real route. All three heuristics are safe here, but the safety argument of Section 5.2 assumed the closed list never locks in a wrong g.

6. When admissible is not enough: consistent heuristics

> **Key point:** A* never expands a cell twice, so a cell closed by a long way in keeps that wrong cost so far g. A heuristic can be admissible and still let that happen. The fix is a consistent heuristic: along any move it never falls by more than the move's length, and then every cell leaves the open list with its shortest g.

Section 5.2 needed the cost so far g of each open cell to be the shortest way in. On the grid that held, because the octile heuristic changes by no more than the move between two neighbouring cells; Section 6.2 shows why that matters. Here is a heuristic that is admissible and still breaks it.

6.1 The closed list throws away a shorter way in

A trace by hand is easier on a few points than on 76 cells, so we use a small roadmap in the same room: five points joined by straight lines through free floor, with lengths in metres. S is the robot at (2.0, 1.0) m, G the doorway's centre at (3.6, 4.0) m, and A, B and C are points of free floor (Figure 6). The shortest route is S-A-C-G:

$$\text{S-A-C-G} = 1.581 + 0.906 + 1.166$$

$$= 3.653 \text{ m}$$

The heuristic is 0 at every point except A, where the robot remembers the exact distance left from an earlier trip:

$$h(A) = 0.906 + 1.166$$

$$= 2.072 \text{ m}$$

No value of the heuristic is above the true distance left, so the heuristic is admissible. Figure 6 runs A* on it; watch frames "run 1: take C off" and "run 1: take A off".

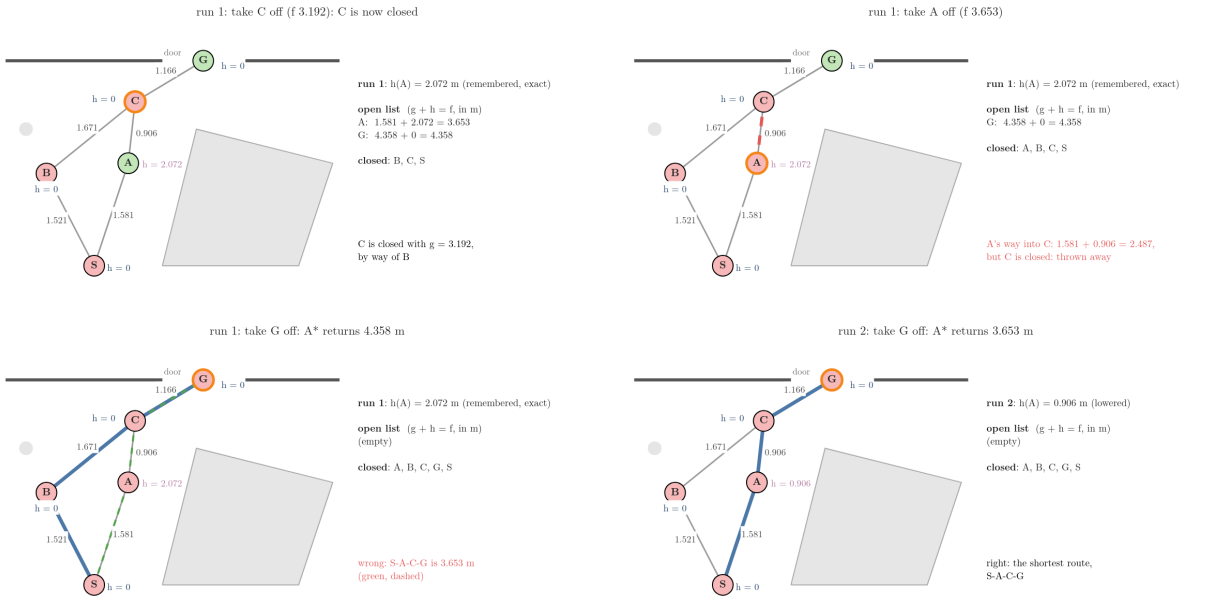


Figure 6: A* with a closed list on a five-point roadmap in the room: S, the robot at (2.0, 1.0) m; G, the doorway's centre at (3.6, 4.0) m; A, B and C, points of free floor; edges labelled with their lengths in m, the heuristic h beside each point. Run 1: h is 0 everywhere except h(A) = 2.072 m, the exact distance left from A. B and then C come off the open list before A, so C is closed with g = 3.192 m by way of B; when A comes off, its way into C, 2.487 m, is thrown away because C is closed (red dashes), and A* returns 4.358 m. The shortest route S-A-C-G is 3.653 m (green, dashed). Run 2: h(A) lowered to 0.906 m; A comes off before C, C gets g = 2.487 m, and A* returns 3.653 m

Step by step, with the estimated total cost $f = g + h$, the cost so far plus the heuristic (Section 4.1), in m (notebook):

Step	Comes off	Open list after it, as $g + h = f$	What happens
1	S	A: 1.581 + 2.072 = 3.653; B: 1.521 + 0 = 1.521	B looks cheaper
2	B	A: 3.653; C: (1.521 + 1.671) + 0 = 3.192	C is reached by way of B
3	C	A: 3.653; G: (3.192 + 1.166) + 0 = 4.358	C goes on the closed list with $g = 3.192$
4	A	G: 4.358	A's way into C, 1.581 + 0.906 = 2.487, is shorter, but C is closed: thrown away
5	G	(empty)	A* returns 4.358 m

The shortest route is 3.653 m, so A* is wrong by

$$4.358 - 3.653 = 0.705 \text{ m}$$

The heuristic at A was right, yet it made A look so far from the door that B's route reached C first, and the closed list locked C's long g in.

6.2 The fix: a guess that never drops faster than the distance

The trouble is the jump in the heuristic between A and C: over an edge of 0.906 m, h falls from 2.072 m to 0. A real distance left cannot fall by more than the distance moved, so a heuristic that does breaks the picture A* relies on. With:

- n, a cell or point;
- n', a neighbour of it, one move away;
- c, the length of the move from n to n', in m.

The rule is, for every move:

$$h(n) \leq c + h(n')$$

A heuristic that meets this on every move is a **consistent heuristic** (G-2580), also called a monotone heuristic (CMU 16-350, A*; Wikipedia, A* search algorithm). The rule is a triangle inequality: going from n to G directly is estimated no longer than going by way of n'. Figure 7 checks one move of each example.

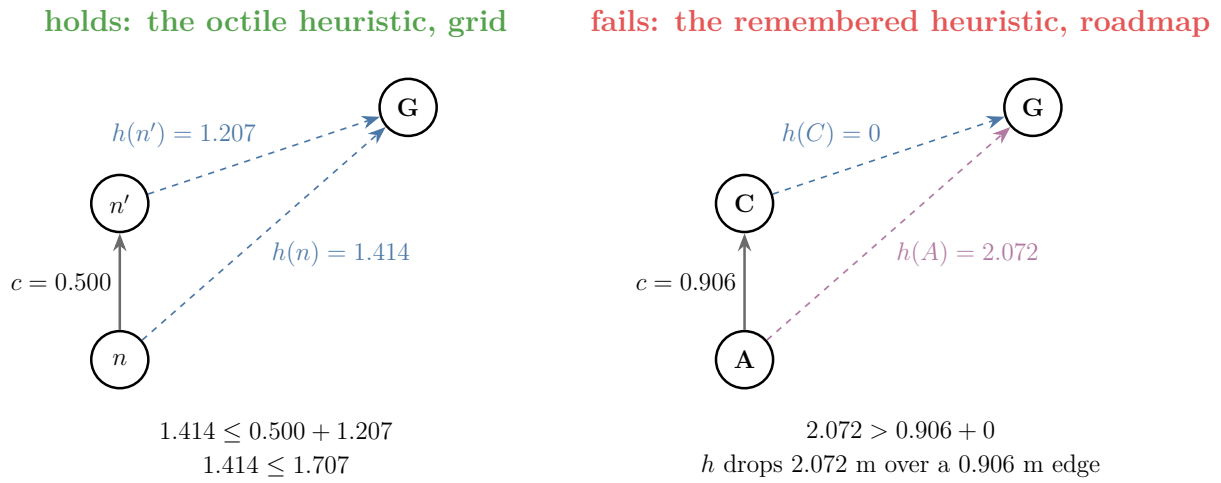


Figure 7: The consistency rule h(n) ≤ c + h(n') checked on one move of each example. Left: on the grid, n is the cell below the table's corner and n' the cell above it; the move is 0.500 m, the octile heuristic values are 1.414 m and 1.207 m, and 1.414 ≤ 1.707: the rule holds. Right: on the roadmap, the move from A to C is 0.906 m, h(A) = 2.072 m and h(C) = 0, and 2.072 > 0.906: the rule fails

On the roadmap the rule fails at A:

$$h(A) = 2.072 > 0.906 + 0$$

Lowering $h(A)$ to 0.906 m makes every edge pass. Run 2 of Figure 6 makes that one change. A's estimated total cost is now

$$f(A) = 1.581 + 0.906$$

$$= 2.487 \text{ m}$$

less than C's 3.192 m by way of B, so A comes off before C. C is then reached by way of A with $g = 2.487$ m before it is closed, and A* returns 3.653 m.

On the grid the octile heuristic passes on every move of the room, and so does the straight line (notebook): one move changes the obstacle-free distance to G by at most the move's length (CMU 16-350, heuristics). Consistency also gives admissibility (CMU 16-350, A*; Wikipedia, Consistent heuristic). Along the shortest route from the cell below the table's corner, n_0, through the cell above it, n_1, and the cell left of G, n_2, to G, the moves are 0.5, 0.707 and 0.5 m:

$$h(n_0) \leq 0.5 + h(n_1)$$

$$\leq 0.5 + 0.707 + h(n_2)$$

$$\leq 0.5 + 0.707 + 0.5 + h(G)$$

$$= 1.707 + 0$$

$$= h^*(n_0)$$

The same steps in symbols, with c_1, c_2, c_3 the three moves:

$$h(n_0) \leq c_1 + h(n_1)$$

$$\leq c_1 + c_2 + h(n_2)$$

$$\leq c_1 + c_2 + c_3 + h(G)$$

$$= h^*(n_0)$$

The last line uses $h(G) = 0$: the moves add up to the shortest route. The converse does not hold: run 1's heuristic is admissible but not consistent.

6.3 Why consistency keeps g right

The consistent rule makes f never fall along a route. Take a move from the cell below the table's corner, n, to the cell above it, n', with:

- $g(n)$, the cost so far at n, 1.707 m;
- $h(n)$, its heuristic, 1.414 m, so $f(n) = 3.121$ m;
- c, the move, 0.5 m;
- $h(n')$, the heuristic at n', 1.207 m.

$$f(n') = 1.707 + 0.5 + 1.207$$

$$= 3.414$$

$$\geq 1.707 + 1.414$$

$$= 3.121$$

$$= f(n)$$

The same steps in symbols:

$$f(n') = g(n) + c + h(n')$$

$$\geq g(n) + h(n)$$

$$= f(n)$$

The second line is the consistent rule with $g(n)$ added to both sides. So the f values along any route never decrease, and A* takes cells off the open list in order of f. A cell reached first by a long way, as C by way of B, has a larger f than every cell on its short way in, so those cells come off first and lower its g before it is closed (CMU 16-350, A*; Wikipedia, A* search algorithm).

On the roadmap, run 1's f falls from 3.653 at A to 2.487 at C by way of A: that fall is what let C close early.

A second fix keeps any admissible heuristic: reopen a closed cell when a shorter way into it turns up, at the price of expanding some cells more than once (CS188 §1.4). The octile heuristic is consistent, so on the grid A* expands each cell once and returns the shortest route. What it still costs is memory: it keeps every cell it has reached.

7. Saving memory: iterative deepening and IDA*

> **Key point:** A* stores every cell it has reached; a depth-first search stores only its current route. Iterative deepening repeats a depth-first search with a growing limit; IDA* puts the limit on f, the cost so far plus the heuristic, so it returns the shortest route while storing at most one route: on the door trip, 17 visits with at most 7 cells in memory.

When A* stopped on the door trip it held 23 cells: 9 on the closed list and 14 on the open list (notebook). The room is small, but a planner over position and heading, or over the joints of an arm, has far more states, and A*'s lists grow with the number of states it reaches (CMU 16-350, heuristics).

The idea. Depth-first search (G-2552), the search that dives along one route and backs up at dead ends, stores only the route it is on: memory that grows with the length of a route, not with the number of cells (LaValle §2.2.2). But it returns whatever route it finds first. The fix is to give the dive a limit and repeat it with a larger limit until a route fits.

The standard term. Depth-first search with a limit on the number of moves, run with the limits 0, 1, 2 and so on until it reaches the goal, is **iterative deepening** (G-2581), also called depth-first iterative deepening (DFID) (LaValle §2.2.2; Korf 1985). Iterative deepening finds the route with the fewest moves, like breadth-first search, with the memory of depth-first search. On the door trip, with the search keeping only its current route and skipping any cell already on it (notebook):

Limit (moves)	0	1	2	3	4	5	6
Cells visited	1	9	58	316	1592	7539	15364

At the limit of 6 moves it reaches G, after 24,879 visits in all, by a route of 6 moves and 3.828 m: the fewest moves, not the shortest length, because moves here have two lengths.

IDA*. Put the limit on the estimated total cost $f = g + h$, the cost so far plus the heuristic (Section 4.1), instead of on the number of moves. Round 1 dives from S and drops any branch whose f passes the limit, which starts at $h(S) = 3.121$ m. If no route fits, the next round's limit is the smallest f that passed: no route can be shorter, since f never overestimates.

Depth-first search with a limit on f is **IDA*** (G-2582), iterative-deepening A* (Korf 1985; LaValle §2.2.2). Figure 8 runs it: watch the blue route, the only thing in memory, grow and shrink.

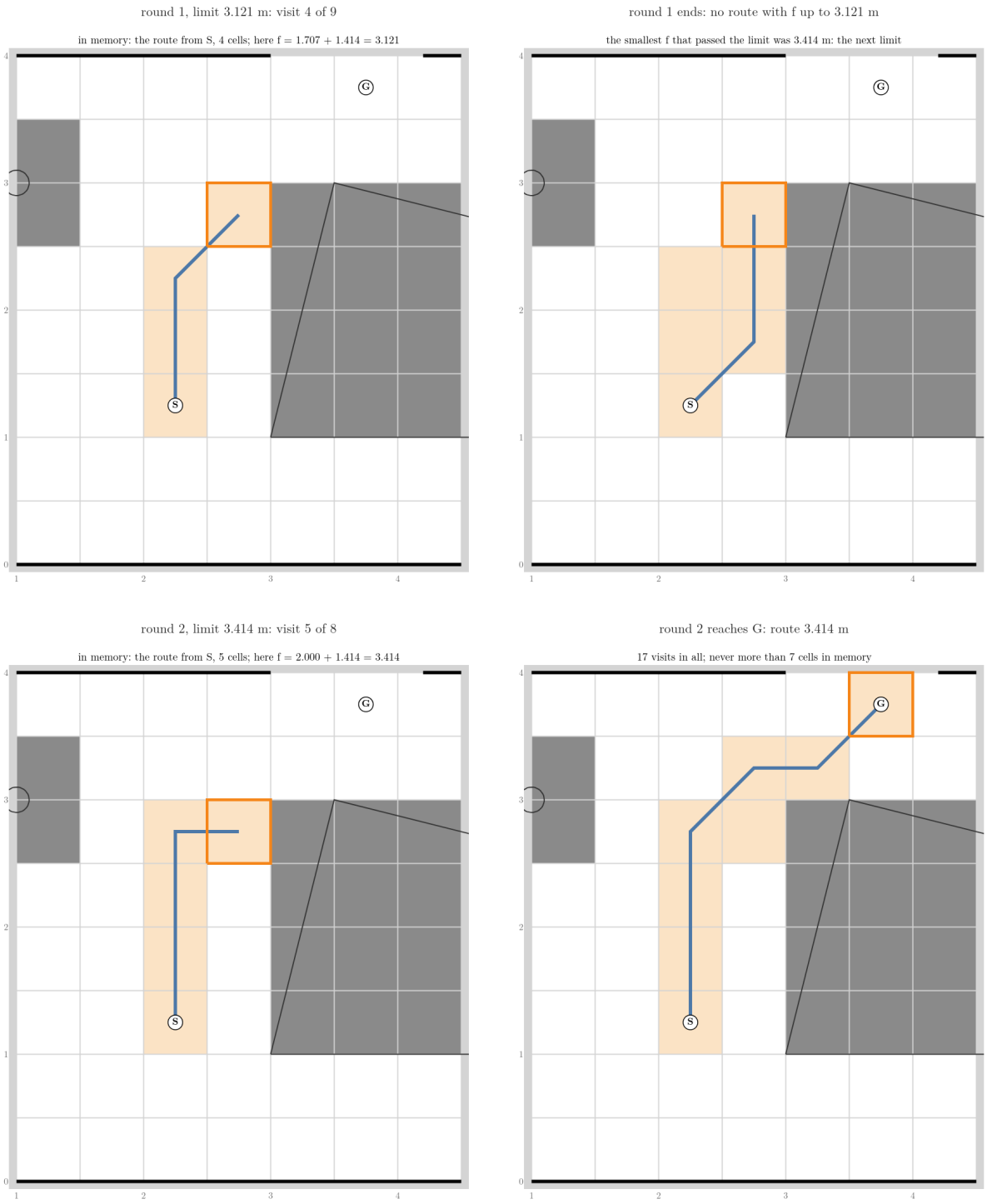


Figure 8: IDA* on the door trip, on the part of the room it touches (x 1.0 to 4.5 m, y 0 to 4 m). Each round is a depth-first search that drops a branch as soon as f = g + h passes the round's limit; the only thing kept in memory is the route from S to the cell being visited (blue). Light orange: cells visited in this round; orange outline: the cell being visited. Round 1, limit 3.121 m (h at S): 9 visits and no route; the smallest f that passed the limit was 3.414 m, the next limit. Round 2, limit 3.414 m: G is reached at the 8th visit, route 3.414 m. 17 visits in all, never more than 7 cells in memory

Round 1 visits 9 cells, some of them twice by different routes, and finds no route with f of 3.121 m or less; the smallest f beyond the limit was 3.414 m. Round 2, limit 3.414 m, reaches G at its 8th visit with a 3.414 m route.

IDA* makes 17 visits against A*'s 9 expansions, with never more than 7 cells in memory against A*'s 23 (notebook): IDA* buys memory with repeated work (Korf 1985). Both searches start at S and run towards G; the next section turns the search round.

8. Searching from the door: backward and bidirectional search

> **Key point:** A search can run from the goal back to the start, or from both ends until they meet. From the door, which stands against the wall, Dijkstra needs 31 cells instead of 54; from both ends, 38. Two searches that meet must still compare every cell both have reached, because the cell where they meet need not lie on the shortest route.

Every search so far started at S. But the door never moves while robots, and the same robot later, start from many places; and the door's side of the room is narrow, against the top wall.

8.1 Backward search

The idea. Run the search from G, following each move backwards, until it reaches S. Our moves cost the same in both directions, so the backward search uses the same moves (LaValle §2.2.3). Its cost so far g at a cell is then the distance left from that cell to G, the true distance left h^* of Section 5.1.

Run to the end, one backward search gives every cell's distance to the door at once: a perfect heuristic for later searches, and the route for any start (CMU 16-350, heuristics). Many robots sharing one destination can use one search from the destination instead of one A* each (Patel, Variants).

The standard term. Searching from the goal towards the start, with every move reversed, is **backward search** (G-2583) (LaValle §2.2.3). Figure 9 (top) runs Dijkstra backwards from G: it reaches S after 31 cells, against 54 forwards (notebook).

Only 30 free cells lie within 3.414 m of G, against 53 within 3.414 m of S, because the door stands against the wall and much of the circle round it lies outside the room. A backward search pays when the goal's side branches less than the start's (LaValle §2.2.3).

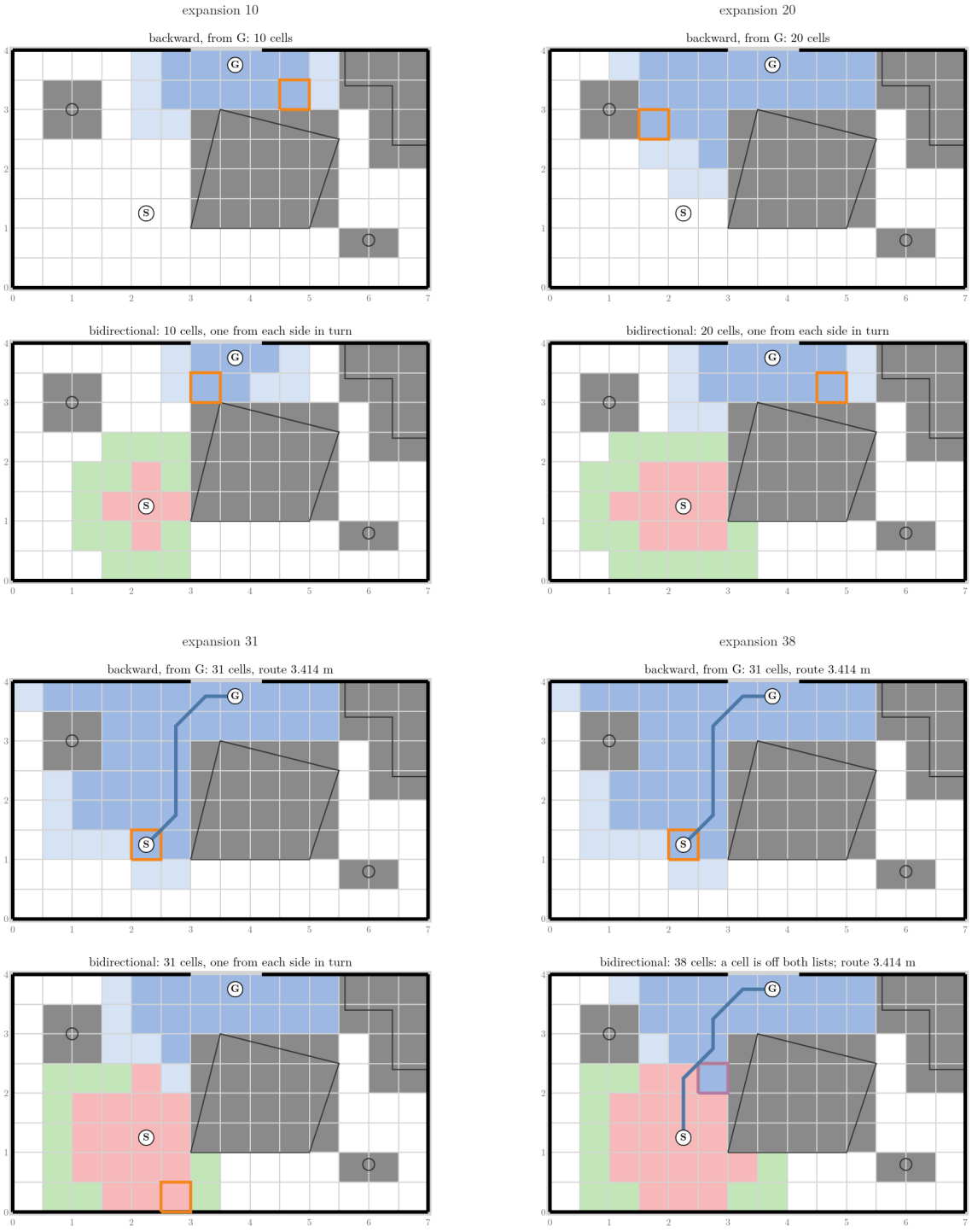


Figure 9: Backward and bidirectional search on the room grid, one expansion per frame in both panels. Top: Dijkstra run backwards from the goal cell G towards S (blue: taken off its open list; pale blue: on it); it reaches S after 31 cells, route 3.414 m. Bottom: bidirectional Dijkstra, one expansion from S (red and green) and one from G (blue) in turn; it stops after 38 cells, when a cell has come off both open lists (purple outline), and its route is 3.414 m. The forward search alone needs 54 cells

8.2 Bidirectional search

The idea. Grow two searches at once, one from S and one from G, a step of each in turn; a route exists as soon as a cell has been reached by both. Each search then has to cover only about half the distance.

The standard term. Running a forward search from the start and a backward search from the goal until they meet is **bidirectional search** (G-2584) (LaValle §2.2.3; MIT 6.006 L18 notes). Figure 9 (bottom) alternates one Dijkstra step from S and one from G: the red ring round S and the blue ring round G grow until a cell has come off both open lists, after 38 cells, 19 from each side.

The 38 are fewer than the 54 of the forward search, but more than the backward search alone: in our room, the door's narrow side is the cheaper one to search from. In many larger problems bidirectional search cuts the exploration sharply (LaValle §2.2.3).

8.3 When to stop, and where the route is

The two searches stop when some cell, call it w, has come off both open lists (MIT 6.006 L18 notes). The route through w seems the obvious answer, but w need not lie on the shortest route. On the door trip the meeting cell happens to lie on a shortest route, so Figure 10 shows the trap on a shorter trip: from R, the cell two above S, to G.

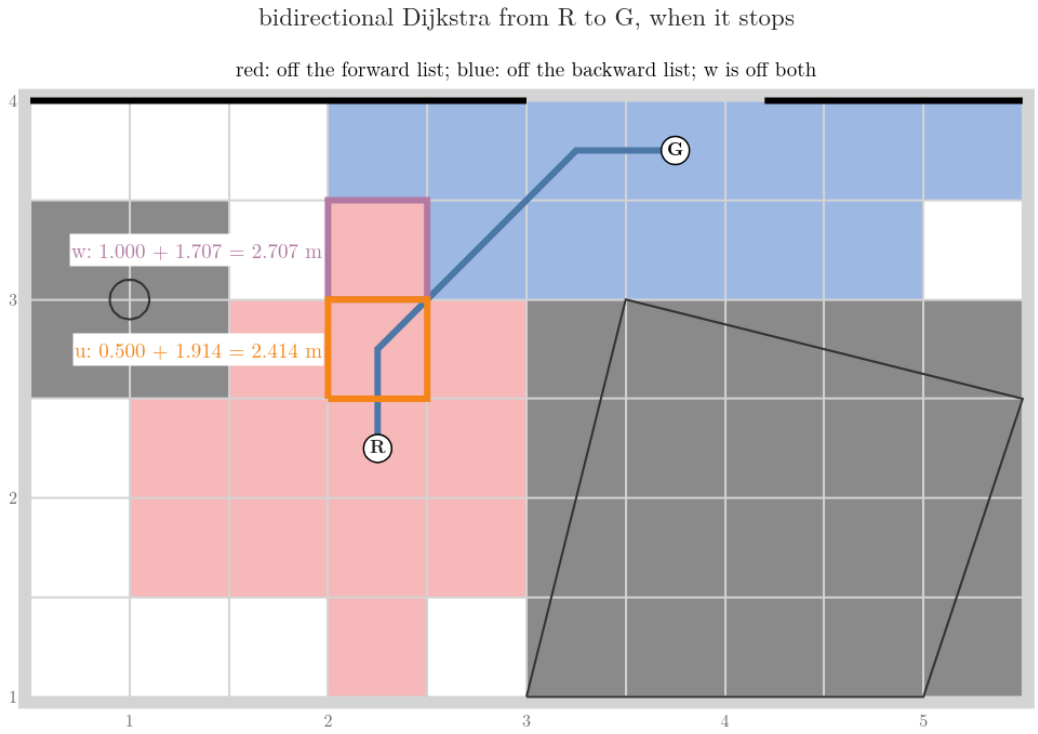


Figure 10: Bidirectional Dijkstra from R (x 2.0 to 2.5 m, y 2.0 to 2.5 m, two cells above the start cell) to G, at the moment it stops, on the part of the room x 0.5 to 5.5 m, y 1.0 to 4.0 m. Red: taken off the forward open list; blue: taken off the backward one. The first cell off both lists, w (purple outline), gives a route of 1.000 + 1.707 = 2.707 m; the cell u just above R (orange outline), reached by both searches but taken off only the forward list, gives 0.500 + 1.914 = 2.414 m, the shortest route (blue line)

With:

- g_f, a cell's cost so far from R in the forward search, in m;
- g_b, its cost so far from G in the backward search, in m.

The cell w came off both lists first, but its route is longer:

$$g_f(w) + g_b(w) = 1.000 + 1.707$$

$$= 2.707 \text{ m}$$

The cell above R, u, came off the forward list early and had been reached by the backward search, but not taken off its list:

$$g_f(u) + g_b(u) = 0.500 + 1.914$$

$$= 2.414 \text{ m}$$

So the stopping test is right, but the answer is the smallest sum $g_f + g_b$ over every cell both searches have reached, not the sum at w (MIT 6.006 L18 notes). On the door trip that smallest sum is 3.414 m. A search from G treats G as one fixed cell; the next section lets the goal be any of several cells.

9. Several goals: multi-goal A*

> **Key point:** When any of several goals will do, join each goal to one added vertex, the virtual goal, and run A* to it once, with the smallest of the guesses to the goals: for three charging docks this takes 12 cells instead of 30 for three separate runs, and ends at the dock nearest by route, 3.207 m away, which is not the dock nearest in a straight line. A goal that moves becomes many goals, one per cell and time: searching over (cell, time), the robot meets a walking person at 5.414 s.

Every search so far ran to the one goal cell G. Now the robot's battery is low, and any of three charging docks will do (Figure 11, first frame):

- G, the cell under the door (x 3.5 to 4.0 m, y 3.5 to 4.0 m);
- D1, the top-left corner cell (x 0 to 0.5 m, y 3.5 to 4.0 m);
- D2, the cell below the table's right end (x 5.0 to 5.5 m, y 0.5 to 1.0 m), beside the green pole.

The robot wants the dock with the shortest route, and the straight line does not tell which one that is. Running A* once per dock does (notebook):

Dock	Straight line from S (m)	A* route from S (m)	Cells taken off the open list
G	2.915	3.414	9
D1	3.202	3.914	14
D2	3.041	3.207	7

G is the nearest in a straight line, but its route bends round the table; the route to D2 runs almost straight, along the row below the table, so D2 is the nearest by route. The three runs together take

$$\text{cells} = 9 + 14 + 7$$

$$= 30$$

and every extra dock would add one more search.

9.1 The virtual goal

The idea. The three runs search the same room three times; one search can find the nearest dock instead. Add one new vertex V and join each dock to it by an edge of 0 m (CMU 16-350, A*). V is not a place in the room; reaching it stands for "docked". Every route to V passes through one dock and costs exactly as much as the route to that dock, so the shortest route to V ends at the dock nearest by route (Patel, Heuristics).

The guess. A* to V needs a heuristic: a guess of the distance left to V, which is the distance to whichever dock is nearest. The guess is the smallest of the octile distances of Section 2 to the three docks (Patel, Heuristics). D1 is 4 columns and 5 rows from S, D2 is 6 columns and 1 row, so with:

- h_{G}, h_{D1}, h_{D2}, the octile distances from a cell to G, D1 and D2, in m;
- h, the heuristic of the search to V at that cell, in m.

$$h_{\mathrm{G}}(S) = 3.121 \text{ m}$$

$$h_{\mathrm{D1}}(S) = 4 \times 0.707 + 1 \times 0.5$$

$$= 3.328 \text{ m}$$

$$h_{\mathrm{D2}}(S) = 1 \times 0.707 + 5 \times 0.5$$

$$= 3.207 \text{ m}$$

$$h(S) = \min(3.121, 3.328, 3.207)$$

$$= 3.121 \text{ m}$$

The same in symbols:

$$h = \min(h_{\mathrm{G}}, h_{\mathrm{D1}}, h_{\mathrm{D2}})$$

The guess never overestimates: the shortest route to V ends at some dock, its length is at least the octile distance to that dock, and that distance is at least the smallest of the three.

The standard term. A* run to an added vertex joined to every acceptable goal is **multi-goal A*** (G-2585), and the added vertex is a **virtual goal** (G-2586), also called an imaginary

goal (CMU 16-350, A*). A backward search from V is a backward search from all goals at once (LaValle §2.2.3).

The run. Figure 11, run 1, plays multi-goal A* one cell per frame; each frame's subtitle gives the dock the guess comes from and the estimated total cost f, the cost so far plus the guess, of the cell just taken off. Watch the red cells climb towards the door first, then turn to D2 at frame "cell 7".

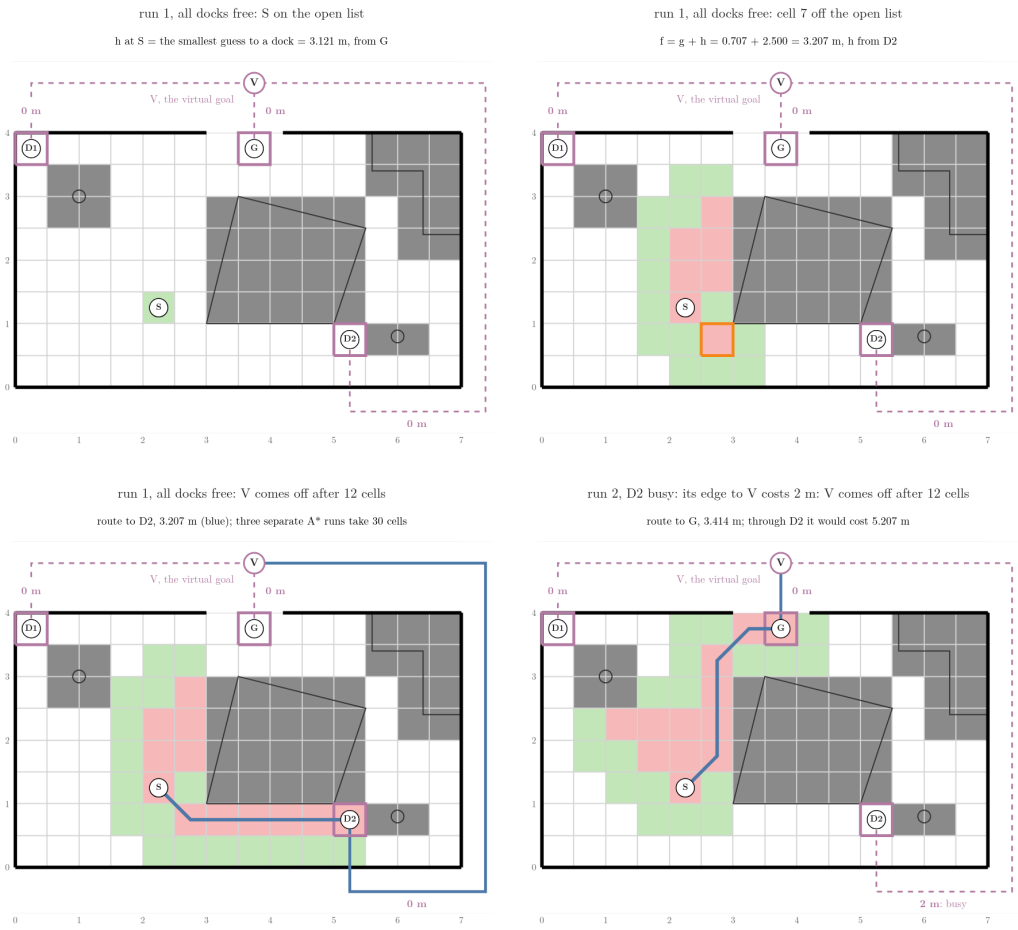


Figure 11: Multi-goal A* to the nearest of three charging docks (purple outlines): G under the door, D1 in the top-left corner, D2 below the table's right end. Each dock has an edge (purple, dashed) to V, the virtual goal, drawn outside the room because V is not a place: the drawn length means nothing, and the edge costs what its label says. A* searches for V with the guess h = the smallest octile distance to a dock (plus that dock's edge), one cell taken off the open list per frame. Light red: cells taken off the open list; light green: cells on it; orange outline: the cell just taken off. Run 1, all edges 0 m: 6 cells climb towards the door, each with f, the cost so far plus the guess, of 3.121 m; from frame "cell 7" the guess comes from D2, and 6 cells run along the row below the table with f of 3.207 m, and V comes off: route to D2, 3.207 m (blue), after 12 cells, where three separate A* runs take 30. Run 2, D2 busy, its edge 2 m: the search also tries 3 cells towards D1 (f of 3.328 m) and ends at G: route 3.414 m after 12 cells

The guess from G is the smallest at S, so the search first climbs towards the door along the table's left side, as in Section 4.2, with f at 3.121 m for 6 cells. At the table's corner the door route must bend and its f rises to 3.414 m. The open cell with the smallest f is now the cell down and to the right of S, on the way to D2 (frame "cell 7"), with:

- g, its cost so far, 0.707 m, one diagonal move;
- h, its guess, 2.5 m, the octile distance to D2, 5 columns away.

$$f = 0.707 + 2.5$$

$$= 3.207 \text{ m}$$

Each cell along the row below the table keeps f at 3.207 m, so A* takes off 5 more cells, the last of them D2, and then V. Multi-goal A* takes 12 cells against the 30 of the three separate runs, and returns the route of the run to D2 alone, 3.207 m (notebook):

$$\text{saved} = 30 - 12$$

$$= 18 \text{ cells}$$

A busy dock. Some goals cost more to use than others, such as a parking space far from the shop (CMU 16-350, A*). Say D2 is busy: the robot would have to wait there, and we count the wait as 2 m of extra driving. The edge from D2 to V then costs 2 m instead of 0 (CMU 16-350, A*), and D2's guess adds the same 2 m, since every route to V through D2 pays it. With:

- c_{D2}, the cost of D2's edge to V, 2 m;
- h_{G}, h_{D1}, h_{D2} and h, the octile distances to the docks and the heuristic, as in the guess above.

$$h(S) = \min(3.121, 3.328, 3.207 + 2)$$

$$= \min(3.121, 3.328, 5.207)$$

$$= 3.121 \text{ m}$$

The same in symbols:

$$h = \min(h_{\mathrm{G}}, h_{\mathrm{D1}}, h_{\mathrm{D2}} + c_{\mathrm{D2}})$$

The best route to V through D2 now costs

$$\text{through D2} = 3.207 + 2$$

$$= 5.207 \text{ m}$$

more than the 3.414 m route to G. Run 2 of Figure 11 makes that one change, and A* ends at G: 3.414 m after 12 cells (notebook). Without the 2 m in D2's guess, the guess would still never overestimate, but A* would take 19 cells: a closer guess takes fewer cells (Section 5.3). The docks stand still; the next goal does not.

9.2 A moving goal

Suppose instead the robot must meet a person who walks under the top wall, along a path the robot knows: the person starts in the cell x 5.0 to 5.5 m and moves one cell (0.5 m) to the left every second, so the person is in the first cell from 0 to 1 s, in the next from 1 to 2 s, and so on. A cell of that walk is a goal only while the person is in it, so the cell alone cannot say whether the robot has arrived in time. The **search state** (G-2562), everything the next moves depend on, must hold the time as well as the cell (LaValle §7.1.1). Each pair of a cell and a time at which the person is in that cell joins the virtual goal V: these are the possible intercept points (CMU 16-350, A*).

The moves. The robot drives at 0.5 m/s. From a state (cell, time) it can:

- move straight, 0.5 m, which adds 1 s;
- move diagonally, 0.707 m, which adds 1.414 s;
- wait one second in its cell, which adds 1 s.

Time only moves forward, so no move leads back to an earlier time (LaValle §7.1.1). A route's cost is its time, so the cheapest route to V is the earliest meeting.

The guess. The cost is now time, so the cost so far, the guess and their sum are in seconds. With:

- g, the cost so far of a state: the time at which the robot gets there, in s;
- d, the octile distance from the state's cell to the nearest cell of the person's walk, in m (2.5 m from S, straight up);
- v, the robot's speed, 0.5 m/s;
- h, the guess of the time left, in s;
- f, their sum, the estimated total cost, in s.

$$h(S) = 2.5/0.5$$

$$= 5 \text{ s}$$

The same in symbols:

$$h = d/v$$

The guess never overestimates: every meeting happens in a cell of the walk, and the robot cannot get there faster.

The run. A* over (cell, time) takes 8 states off the open list, then V (notebook). The first 6 are the straight line up from S, at 0 to 5 s, each with f of 5 s: driving straight at the top wall adds to g exactly what it takes off h. At 5 s the robot would be in the cell x 2.0 to 2.5 m under the wall, but the person is then in the cell to its right. The cheapest states left have f of 5.414 s, and ties go to the smaller guess (Section 4.2), so the first two are the diagonals from the cell below into the top row: into x 1.5 to 2.0 m, where nobody is, and into x 2.5 to 3.0 m, where the person is from 5 to 6 s. The route is four cells straight up and then that diagonal, with:

- L, the length of the route, in m;
- T, the time at which the robot reaches the meeting cell, in s.

$$L = 4 \times 0.5 + 0.707$$

$$= 2.707 \text{ m}$$

$$T = 2.707/0.5$$

$$= 5.414 \text{ s}$$

The same in symbols:

$$T = L/v$$

The person is in that cell until 6 s, so they meet at 5.414 s, and the robot never waits (notebook). Figure 12 plays the meeting.

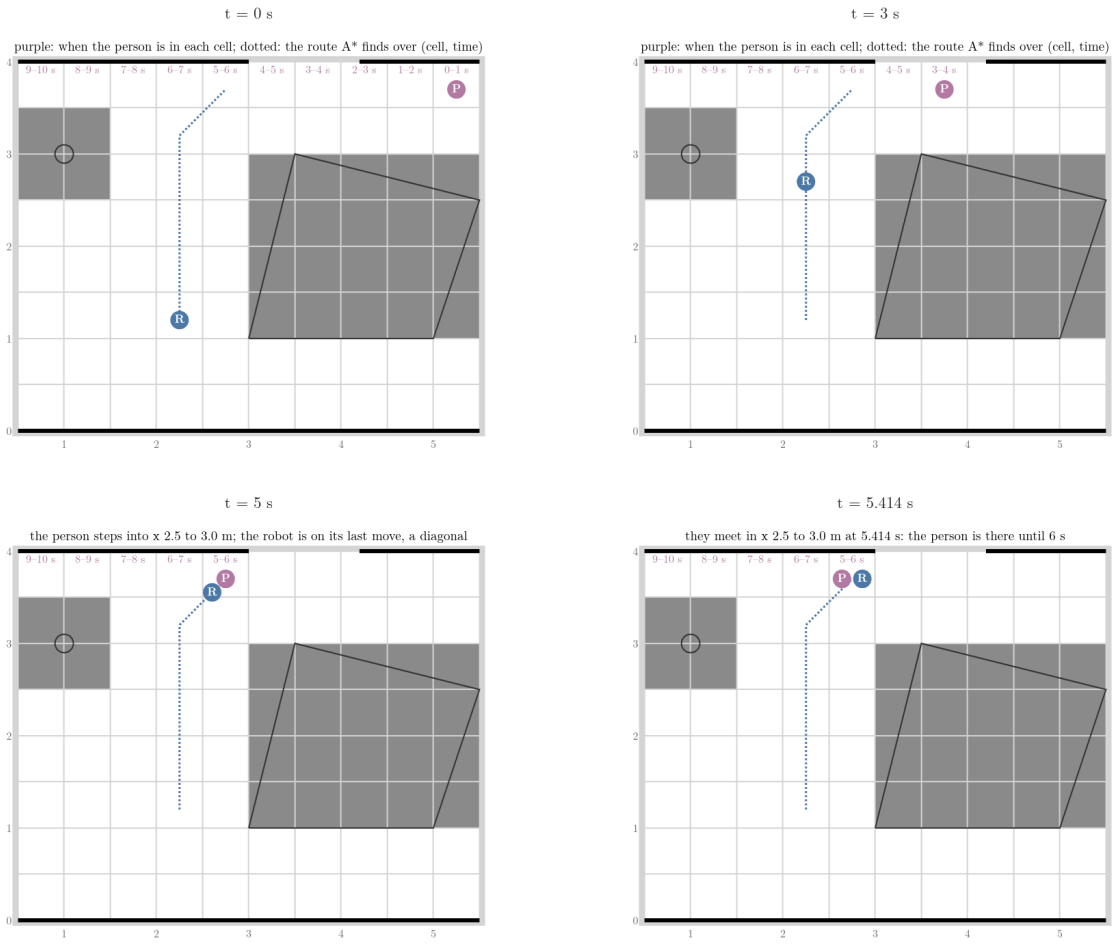


Figure 12: A moving goal, on the part of the room x 0.5 to 5.5 m: a person P walks under the top wall from x 5.0 to 5.5 m towards the left, one 0.5 m cell per second; purple labels give the seconds the person is in each cell, and each such cell and time is a goal. The robot R drives at 0.5 m/s along the dotted route that A* over (cell, time) finds: four cells straight up, then a diagonal. Frames at 0, 1, 2, 3, 4 and 5 s, and at 5.414 s, when the robot enters the cell x 2.5 to 3.0 m while the person is in it (5 to 6 s): they meet

A check, cell by cell: the robot's earliest arrival in a cell of the walk is its shortest route there divided by 0.5 m/s, and the robot meets the person there if it arrives before the person leaves (notebook):

Person's cell (x, m)	Person in it (s)	Robot's shortest route (m)	Robot there at (s)	Meet at (s)
3.5 to 4.0 (G)	3 to 4	3.414	6.828	too late
3.0 to 3.5	4 to 5	2.914	5.828	too late
2.5 to 3.0	5 to 6	2.707	5.414	5.414
2.0 to 2.5	6 to 7	2.5	5.0	6, after waiting 1 s

The earliest meeting is the one A* found; the last row is where the wait move matters, since the robot arrives first and waits. The table works here because nothing else in the room moves; when obstacles move too, a cell is free only at some times, and the search over (cell, time) handles that the same way (LaValle §7.1.1).

Multi-goal search changes what counts as arriving. On a large map, even one A* run can take longer than the robot can wait.

10. Trading length for time: weighted A* and anytime A*

> **Key point:** Multiplying the heuristic by a weight ε above 1 makes A* rush towards the goal: fewer cells, and a route at most ε times the shortest. Anytime A* runs weighted A* with falling weights and always has a route ready: on the long trip, 9.243 m at once, then 8.414 m, then the shortest, 8.000 m.

A robot must often start moving before an exact search finishes, and a slightly longer route found at once is better than the best route found too late (CMU 16-350, anytime). The door trip is too easy to show the trade: every weight there returns the same 3.414 m. So we use the long trip of Section 3, where A* takes 43 cells to find 8.000 m.

10.1 Weighted A*

The idea. Between Dijkstra, which ignores the heuristic, and greedy search, which follows nothing else, lies a whole range: count the heuristic more than the distance driven. With:

- ε, the weight on the heuristic, a number of at least 1 (1 gives A*);
- g, h, the cost so far and the heuristic of Section 4.1, in m.

A* takes off the open list the cell with the smallest

$$f = g + \varepsilon \times h$$

which is **weighted A*** (G-2587) (CMU 16-350, heuristics; Pohl 1970). A large ε behaves like greedy search; $\varepsilon = 1$ is A* (CMU 16-350, heuristics). On the long trip (notebook):

ε	Cells taken off the open list	Route (m)	At most $\varepsilon \times$ 8.000 m
2.5	22	9.243	20.000
1.2	38	8.414	9.600
1.0	43	8.000	8.000

Why the route is at most ϵ times the shortest. The chain of Section 5.2 loosens by the weight. Take the run with $\varepsilon = 2.5$ when G comes off, and the cell right of S on the bottom wall, on the shortest route and still on the open list, with:

- n, that cell;

- $g(n)$, 0.5 m, already the shortest way to it;
- $h(n)$, its octile heuristic, 6.6213 m;
- $h^*(n)$, its true distance left, 7.5 m;
- L^*, the shortest route, 8.000 m;
- $g(G)$, the route weighted A* returns, 9.243 m.

$$g(G) = 9.243$$

$$\leq 0.5 + 2.5 \times 6.6213$$

$$= 0.5 + 16.553$$

$$= 17.053$$

$$\leq 2.5 \times 0.5 + 2.5 \times 7.5$$

$$= 2.5 \times 8.0$$

$$= 20.0$$

The same steps in symbols:

$$g(G) \leq g(n) + \varepsilon \times h(n)$$

$$\leq \varepsilon \times g(n) + \varepsilon \times h^*(n)$$

$$= \varepsilon \times L^*$$

The first step holds because G came off before n; the second because $\varepsilon \geq 1$ and the heuristic is admissible. So weighted A* always returns a route at most ε times the shortest (CMU 16-350, heuristics). In practice the route is often much better than the bound:

$$9.243/8.000 = 1.155$$

times the shortest, not 2.5.

10.2 Anytime A* and ARA*

The idea. Run weighted A* with a large ε to get a route at once, and then again with smaller and smaller ε, each run giving a better route, until $\varepsilon = 1$ gives the shortest. The robot can stop the planner whenever it must move and use the best route so far (CMU 16-350, anytime).

The standard terms. A search that can return a route whenever it is interrupted and improves it while it has time is an **anytime search** (G-2589) (CMU 16-350, anytime).

Three fresh weighted A* runs repeat much of the same work. **ARA*** (G-2590), anytime repairing A*, keeps the g values of the last run and re-expands only the cells whose g has dropped since (Likhachev et al. 2003; CMU 16-350, anytime). Figure 13 runs ARA* on the long trip with $\varepsilon = 2.5$, 1.2 and 1.0.

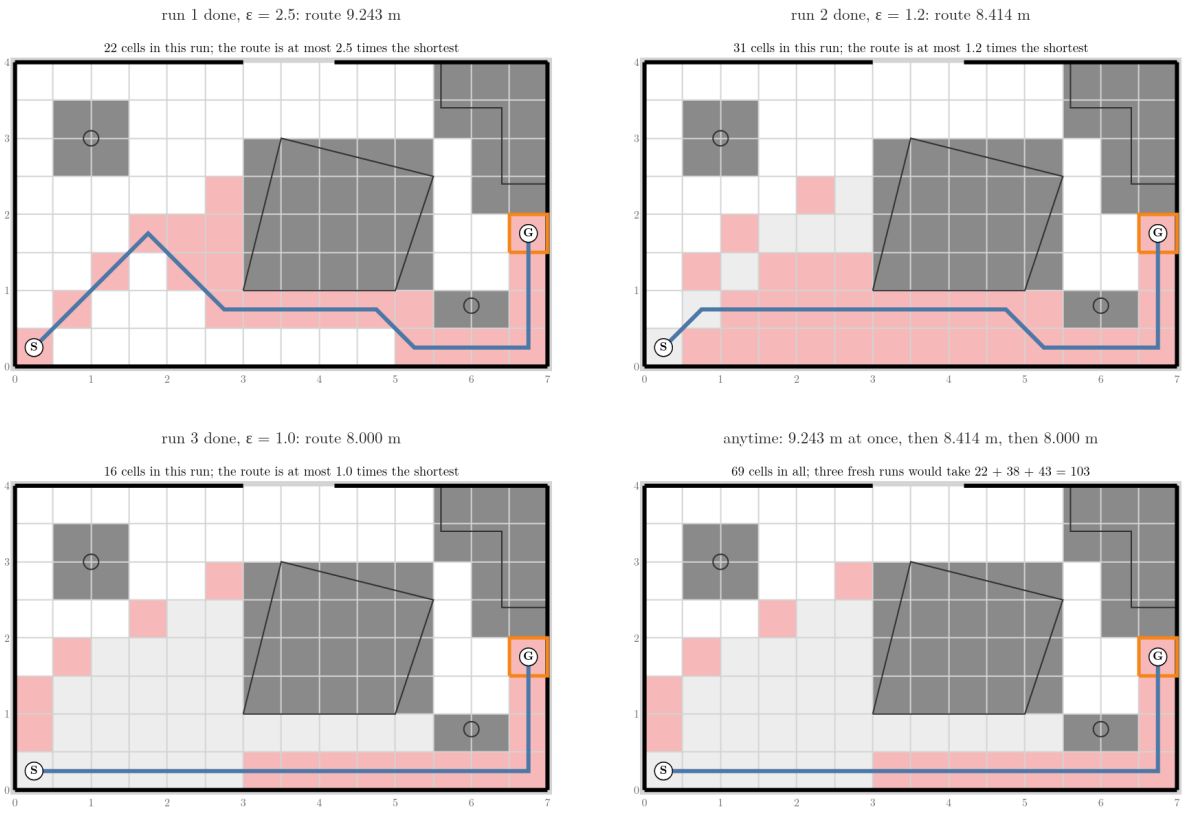


Figure 13: Weighted and anytime A* on the long trip (S in the bottom-left corner cell, G at the right wall), as ARA* runs it: weighted A* with f = g + ϵ h, $\epsilon = 2.5$, then 1.2, then 1.0, each run reusing the g values of the run before. One expansion per frame; red: taken off the open list in this run; grey: in earlier runs; blue: the best route so far. Run 1 (ϵ 2.5): 22 cells, route 9.243 m. Run 2 (ϵ 1.2): 31 more cells, 8.414 m. Run 3 (ϵ 1.0): 16 more cells, 8.000 m, the shortest. 69 cells in all, against 22 + 38 + 43 = 103 for three fresh runs

The route improves from 9.243 m to 8.414 m to 8.000 m, and ARA* takes 69 cells off its

open lists in all, against 103 for three fresh runs (notebook). Weighted and anytime A* save cells by accepting longer routes; the next variant saves cells without giving up any length.

11. Skipping equal routes: jump point search

> **Key point:** On a grid where every move of one kind costs the same, many routes tie, and A* puts cells of all of them on its open list. Jump point search scans along straight lines and diagonals and puts on the open list only the cells where a route may have to turn: on the door trip, 7 cells ever enter its open list, against 23 for A*, for the same 3.414 m route.

In Figure 1, Dijkstra and A* returned two different routes of the same 3.414 m. On a grid with no obstacles in the way, the number of such equal routes between two cells grows fast, and A* treats each one as a separate possibility, putting its cells on the open list and keeping their numbers.

The idea. Equal routes differ only in the order of the same moves, such as up then up-right against up-right then up (zerowidth JPS). So from a cell, keep going in one direction for as long as nothing could make a turn necessary, without putting the cells passed on the open list. Stop only at the goal, or next to an obstacle's corner, where a cell beside the line has just opened up and a route may have to turn into it.

The standard terms. A neighbour that the cell before could normally reach by an equally short route of its own, but which an obstacle beside the line leaves reachable by a shortest route only by turning at the current cell, is a **forced neighbour** (G-2592): the search is forced to look at it there.

A cell where a scan stops, because it is the goal or has a forced neighbour, is a **jump point** (G-2593), and A* that puts only jump points on its open list is **jump point search** (G-2594), JPS (Harabor and Grastien 2011; zerowidth JPS).

A diagonal scan checks a straight scan in both of its directions at every step, and stops where one of them finds a jump point (zerowidth JPS). Our grid forbids cutting corners, so we use the version for that rule (Harabor and Grastien 2012). Figure 14 runs it on the door trip.

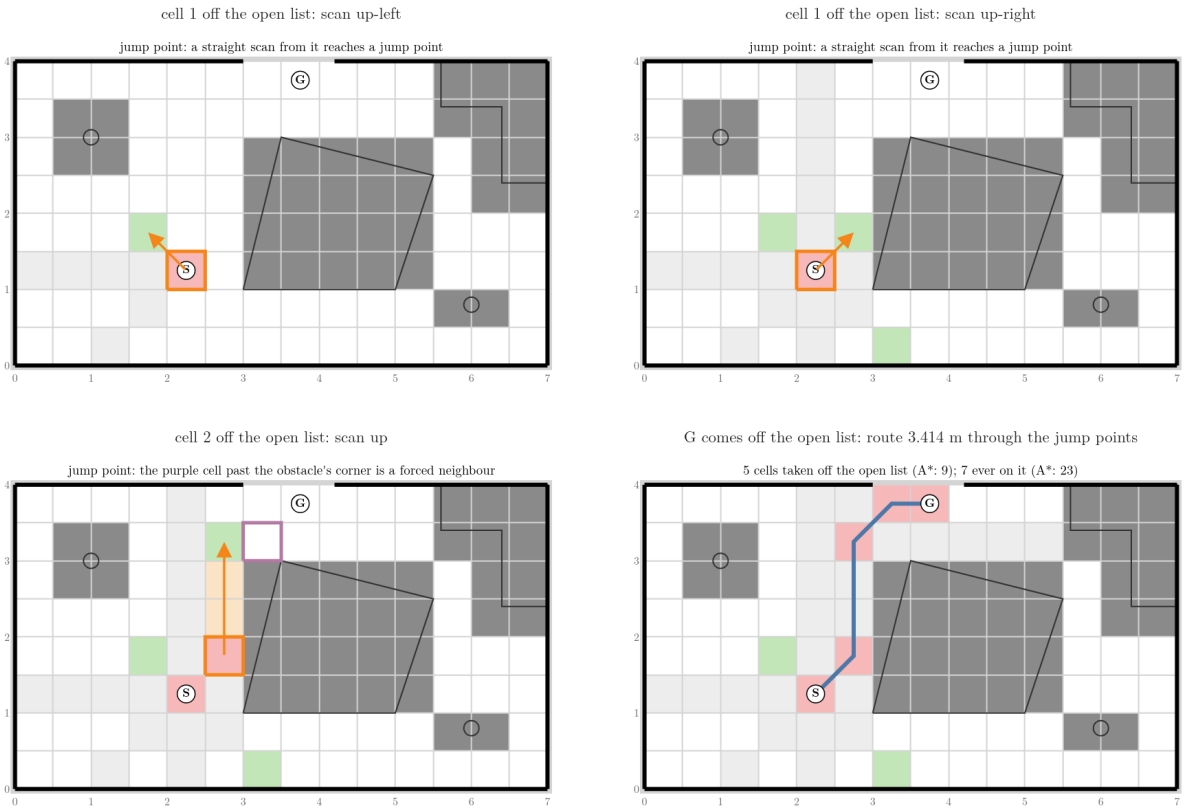


Figure 14: Jump point search on the room grid from S to G, one scan per frame. From each cell it takes off the open list, it scans in straight lines and diagonals without putting the cells it passes on the open list (light orange: this scan; grey: earlier scans); a scan stops at a wall or obstacle (nothing found) or at a jump point (green: on the open list; red: taken off it): G, or a cell with a forced neighbour (purple outline), a cell past an obstacle's corner where a route may have to turn. Scanning up from the cell up-right of S stops beside the table's corner, where the cell to the right opens up. JPS takes 5 cells off the open list (A*: 9) and only 7 cells ever enter it (A*: 23); the route through the jump points is 3.414 m

Watch frame "cell 2 off the open list: scan up": the scan from the cell up-right of S runs up the table's left side and stops beside the table's corner, where the cell to its right (purple outline) has just come free. The route turns there. JPS takes 5 cells off its open list, against A*'s 9, and only 7 cells ever enter it, against A*'s 23 (notebook); on the long trip it takes 6 cells off against A*'s 43.

The scans still look at cells, but looking costs far less than putting a cell on the open list and taking it off again, so JPS is faster without giving up the shortest route, on grids where every straight move and every diagonal move costs the same (Harabor and Grastien 2011; zerowidth JPS). Its route is still a chain of grid moves at 0° or 45°.

12. Routes at any angle: Theta*

> **Key point:** A grid route can only turn by multiples of 45°, so it is longer than a straight line through free space. Theta* lets a cell take its parent's parent as its own parent whenever the straight line between them is clear, which shortens the door route from 3.414 m to 3.193 m.

Every route so far, from Dijkstra's to JPS's, is made of moves along the grid: straight or at 45°. The 3.414 m route of Section 4 goes up-right, up, up, up, up-right, right, while a robot could drive one straight line from S to beside the door and then turn.

The idea. A* builds every route from parent links between neighbouring cells. If a cell's parent can see the cell's neighbour directly, along a straight line that crosses no blocked cell, the neighbour can link to that parent instead, and its route becomes one straight segment instead of a bend (Daniel et al. 2010).

The standard terms. Two cells can see each other when the straight segment between their centres crosses no blocked cell. Search whose routes may run at any angle between cell centres, not just along grid moves, is **any-angle search** (G-2596), and **Theta*** (G-2597) is A* with the shortcut above (Daniel et al. 2010; Patel, Variants). Figure 15 runs it on the door trip; most cells end up linked straight to S.

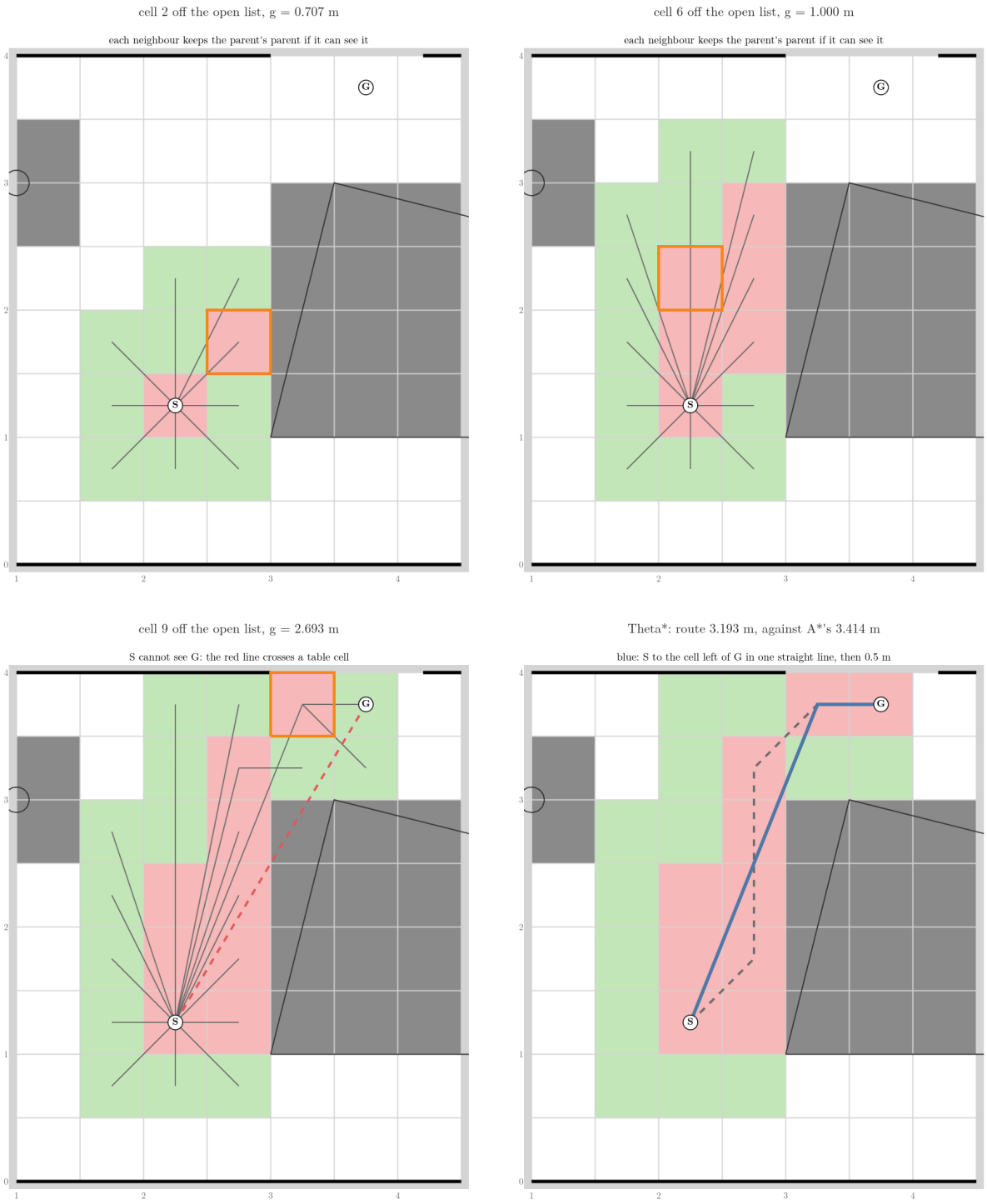


Figure 15: Theta* on the door trip, on the part of the room it touches (x 1.0 to 4.5 m, y 0 to 4 m). Theta* is A* with one change: when a cell is taken off the open list, each neighbour first tries the cell's own parent as its parent, and keeps it if the straight line between their centres crosses no blocked cell, so the two cells can see each other. Grey lines join each cell to its parent; most cells see S directly. One expansion per frame. When the cell left of G is expanded, S cannot see G (the red dashed line crosses a table cell), so G's parent is that cell. Last frame: the Theta* route, 2.693 + 0.500 = 3.193 m (blue), against the grid route of A*, 3.414 m (grey, dashed)

The Theta* route is one straight segment from the centre of S to the centre of the cell left of G, 1.0 m across and 2.5 m up, then one 0.5 m move into G:

$$\text{segment} = \sqrt{1.0^2 + 2.5^2}$$

$$= \sqrt{7.25}$$

$$= 2.693 \text{ m}$$

$$\text{route} = 2.693 + 0.5$$

$$= 3.193 \text{ m}$$

The Theta* route is shorter than the grid route by

$$3.414 - 3.193 = 0.221 \text{ m}$$

S cannot see G itself, because that line crosses a table cell (frame “cell 9 off the open list”). Theta* does not promise the shortest route at any angle (Daniel et al. 2010): a taut string from S round the corner of the blocked cell at (3.0 m, 3.0 m) to G would be 2.965 m (notebook).

13. Summary

Search	Takes off the open list the cell with the smallest	On the door trip	Why use it
Dijkstra	g	54 cells, 3.414 m	no heuristic needed
greedy best-first	h	7 cells, 3.414 m (long trip: 9.536 m against 8.000 m)	few cells, no guarantee
A*	$f = g + h$	9 cells, 3.414 m	shortest route with an admissible, consistent heuristic
IDA*	f, depth-first under a limit	17 visits, 7 cells in memory	little memory
backward / bidirectional	g from G / from both ends	31 / 38 cells	goal side narrow; many starts
multi-goal A*	f to a virtual goal	three docks: 12 cells (30 for three runs), 3.207 m	any of several goals
weighted / ARA*	$g + \varepsilon h$	long trip: 9.243, 8.414, 8.000 m	a route at once, improved with time
jump point search	f, jump points only	5 cells off, 7 on the open list	open grids with equal move costs
Theta*	f, with parents a cell can see	3.193 m	routes at any angle

Here g is the cost so far from S, h the heuristic, the guess of the distance left, $f = g + h$, and ε the weight on the heuristic.

- The robot's search to the door no longer spreads all round it: A* takes 9 cells off its open list instead of Dijkstra's 54 and still returns the 3.414 m route, because it adds to the distance driven g a heuristic h, a guess of the distance left, so cells away from the door rank lower.
- A **heuristic** (G-2570) is a cheap guess of the cost left; the **octile distance** (G-2571), 3.121 m from S, is the exact distance on the same grid without obstacles, so it can be worked out by counting columns and rows.
- **Greedy best-first search** (G-2573) follows the heuristic alone, so it looks at few cells but can commit to a detour: 9.536 m on the long trip where 8.000 m exists.
- **A*** (G-2577) ranks cells by the **estimated total cost** (G-2575) f, the **cost so far** (G-2574) g plus h, and keeps a parent (G-2576) for each cell, so following parents back from G gives the route.
- An **admissible heuristic** (G-2578) never overestimates, so f is a lower bound on any route through a cell and the first route out of the open list is the shortest.
- The heuristic is the one free choice in A*: with the loop unchanged, a **dominating**

(G-2579) admissible heuristic, closer to the true distance left, takes fewer cells, because fewer cells' f stay at or below the route's length: 54 for $h = 0$, 11 for the straight line, 9 for the octile distance, all with the 3.414 m route.

- A **consistent heuristic** (G-2580) never falls by more than a move's length, so f never decreases along a route and the closed list never locks in a long g; the remembered heuristic on the roadmap was admissible but not consistent and returned 4.358 m instead of 3.653 m.
- **Iterative deepening** (G-2581) and **IDA*** (G-2582) repeat depth-first searches with a growing limit, so they keep only one route in memory, 7 cells against A*'s 23, at the price of repeated visits, 17 against 9.
- **Backward search** (G-2583) from the door needs 31 cells because the door's side is narrow, and gives every cell's true distance to the door; **bidirectional search** (G-2584) needs 38, and must take the smallest sum over all cells both searches reached, because the meeting cell can be off the shortest route (2.707 m against 2.414 m).
- **Multi-goal A*** (G-2585) joins every acceptable goal to a **virtual goal** (G-2586), so one search finds the dock nearest by route, 3.207 m away, after 12 cells instead of 30 for three searches; a search over (cell, time) turns a walking person into many goals, met at 5.414 s.
- **Weighted A*** (G-2587) multiplies the heuristic by ε, so it takes fewer cells for a route at most ε times the shortest; **anytime search** (G-2589) such as **ARA*** (G-2590) lowers ε run by run and reuses the work, so a 9.243 m route is ready at once and the 8.000 m one after 69 cells instead of 103.
- **Jump point search** (G-2594) skips equal routes that differ only in the order of their moves by scanning to **jump points** (G-2593), cells with a **forced neighbour** (G-2592), so only 7 cells enter its open list against A*'s 23.
- **Theta*** (G-2597), an **any-angle search** (G-2596), takes a parent's parent wherever the two cells can see each other, so the door route becomes 3.193 m instead of 3.414 m.

The robot can now plan its shortest route to the door quickly, with variants for little memory, many goals, little time and routes at any angle. Still missing: every search here moves between places in a grid, while a robot's task is often a sequence of actions, such as picking up a cup and carrying it out of the door; symbolic task planning (G-2600) searches over facts about the world instead of cells.

14. Sources

Built from

- Sebastian Lague, "A* Pathfinding (E01: algorithm explanation)", YouTube, https://www.youtube.com/watch?v=-L-WgKMFuhE
- MIT OpenCourseWare, "5. Search: Optimal, Branch and Bound, A*" (MIT 6.034 Artificial Intelligence), YouTube, https://www.youtube.com/watch?v=gGQ-vAmdAOI (MIT 6.034)
- MIT OpenCourseWare, "18. Speeding up Dijkstra" (MIT 6.006 Introduction to Algorithms), YouTube, https://www.youtube.com/watch?v=CHvQ3q_gJ7E, and its lecture notes, https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-

2011/resources/mit6_006f11_lec18/ (MIT 6.006 L18 notes)

- NPTEL, *Artificial Intelligence: Search Methods for Problem Solving* (IIT Madras), "8. Search DFID", YouTube, https://www.youtube.com/watch?v=xIEe0yK1VL4, "9. Heuristic Search", YouTube, https://www.youtube.com/watch?v=dtGRmhZ6Cuo, and "21. A* Monotone Property, Iterative Deepening A*", YouTube, https://www.youtube.com/watch?v=a (NPTEL AI search, lectures 8, 9 and 21)
- UC Berkeley CS188, *Introduction to Artificial Intelligence*, online textbook, §1.4 "Informed Search", https://inst.eecs.berkeley.edu/~cs188/textbook/search/informed.html (CS188 §1.4)
- Likhachev, M. (Carnegie Mellon University), *16-350 Planning Techniques for Robotics*, slides "Search Algorithms: A* Search, Multi-goal A*", https://www.cs.cmu.edu/~maxim/classes/robot (CMU 16-350, A*); "Search Algorithms: Heuristics, Backward A*, Weighted A* Search", https://www.cs.cmu.edu/~maxim/classes/robotplanning/lectures/heuristicsweightedsearch_16350_sp (CMU 16-350, heuristics); "Interleaving Planning and Execution: Anytime Heuristic Search", https://www.cs.cmu.edu/~maxim/classes/robotplanning/lectures/anytimesearch_16350_sp2 (CMU 16-350, anytime)
- Patel, A. *Amit's A* Pages*, "Heuristics", https://theory.stanford.edu/~amitp/GameProgramming/Heu (Patel, Heuristics), and "Variants of A*", https://theory.stanford.edu/~amitp/GameProgramming/Vari (Patel, Variants)
- zerowidth.com, "A Visual Explanation of Jump Point Search" (2013), https://zerowidth.com/2013/05/0 point-search-explained.html (zerowidth JPS)
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. §2.2.2 "Particular forward search methods" (A*, iterative deepening and IDA*), §2.2.3 "Other general search schemes" (backward and bidirectional search) and §7.1.1 "Problem formulation" (time-varying problems: a state holds the configuration and the time, and time only moves forward). Free online: https://lavalle.pl/planning/node45.html, https://lavalle.pl/planning/node47.html, https://lavalle.pl/planning/node49.html, https://lavalle.pl/planning/node50.html and https://lavalle.pl/planning/node312.html (LaValle §2.2.2, §2.2.3, §7.1.1)

Other references

- Wikipedia, "A* search algorithm": straight-line distance as a heuristic that never overestimates, and the consistent (monotone) rule, https://en.wikipedia.org/wiki/A*_search_algorithm (Wikipedia, A* search algorithm)
- Wikipedia, "Consistent heuristic": a consistent heuristic is also admissible, but not the other way round, https://en.wikipedia.org/wiki/Consistent_heuristic (Wikipedia, Consistent heuristic)
- Hart, P. E., Nilsson, N. J. and Raphael, B. (1968). "A formal basis for the heuristic determination of minimum cost paths". *IEEE Transactions on Systems Science and Cybernetics* 4(2), 100–107 (Hart et al. 1968)
- Pohl, I. (1970). "Heuristic search viewed as path finding in a graph". *Artificial Intelligence* 1(3), 193–204 (Pohl 1970)
- Korf, R. E. (1985). "Depth-first iterative-deepening: an optimal admissible tree search". *Artificial Intelligence* 27(1), 97–109 (Korf 1985)
- Likhachev, M., Gordon, G. and Thrun, S. (2003). "ARA*: Anytime A* with provable

bounds on sub-optimality". *Advances in Neural Information Processing Systems 16* (Likhachev et al. 2003)
- Harabor, D. and Grastien, A. (2011). "Online graph pruning for pathfinding on grid maps". *Proceedings of AAAI-11* (Harabor and Grastien 2011)
- Harabor, D. and Grastien, A. (2012). "The JPS pathfinding system". *Proceedings of the Symposium on Combinatorial Search (SoCS 2012)*: JPS modified for grids that forbid corner-cutting moves, https://ojs.aaai.org/index.php/SOCS/article/view/18254 (Harabor and Grastien 2012)
- Daniel, K., Nash, A., Koenig, S. and Felner, A. (2010). "Theta*: Any-angle path planning on grids". *Journal of Artificial Intelligence Research* 39, 533–579, https://arxiv.org/abs/1401.3843 (Daniel et al. 2010)

15. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Heuristic h (G-2570)	A cheap guess of the cost still to go from a state to the goal, such as 3.121 m from the start cell to the door; a search uses it to prefer states that look nearer the goal.
Octile distance (G-2571)	The length of the shortest route between two cells of an 8-connected grid with no obstacles: diagonal moves while both the column and the row count shrink, then straight moves; with straight moves of length a and diagonals of length ℓ it is $\ell \min + a(\max - \min)$ of the column and row counts; a heuristic for 8-connected grids that never overestimates.
Greedy best-first search (G-2573)	A search that always expands the open state with the smallest heuristic value, ignoring the cost already paid; it heads straight for the goal and looks at few states, but can return a long route.
Cost so far g (g-value) (G-2574)	The length of the best route found so far from the start to a state; A* lowers it whenever a shorter way into the state turns up.
Estimated total cost f (f-value) (G-2575)	The cost so far plus the heuristic, $f = g + h$: the length of the best route through a state if the guess were exact; A* expands the open state with the smallest f.

Term	Meaning
A* search (G-2577)	Dijkstra's algorithm with the open list sorted by $f = g + h$, the cost so far plus a heuristic guess of the cost left; with a consistent heuristic it returns the shortest route while expanding far fewer states (9 cells against 54 on the room grid).
Admissible heuristic (G-2578)	A heuristic that is never more than the true cost left from any state, $h \leq h^*$; it makes f a lower bound on every route through a state, so the first route A* takes off the open list is the shortest.
Dominance (of heuristics) (G-2579)	One admissible heuristic dominates another when it is at least as large at every state; being closer to the true cost, it makes A* expand fewer states (the octile distance dominates the straight line).
Consistent heuristic (monotone heuristic) (G-2580)	A heuristic that never falls by more than a move's cost, $h(n) \leq c(n, n') + h(n')$, with 0 at the goal (a triangle inequality); it keeps f from decreasing along a route, so A* with a closed list expands every state with its shortest g.
Iterative deepening (DFID) (G-2581)	Depth-first search with a limit on the number of moves, repeated with the limits 0, 1, 2 and so on until the goal is reached; it finds the fewest moves, like breadth-first search, while storing only one route, like depth-first search.
IDA* (iterative-deepening A*) (G-2582)	Iterative deepening with the limit on $f = g + h$ instead of on moves, each new limit being the smallest f that passed the last; it returns the shortest route with an admissible heuristic while storing only the current route.
Backward search (G-2583)	A search that starts at the goal and follows every move backwards until it reaches the start; it is cheaper when the goal's side branches less, and run to the end it gives every state's cost to the goal.
Bidirectional search (G-2584)	A forward search from the start and a backward search from the goal run in turn until some state has been expanded by both; the route is the smallest sum of the two costs over all states both reached, since the meeting state can be off the shortest route.

Term	Meaning
Multi-goal A* (G-2585)	A* run to an added vertex joined to every acceptable goal, so one search finds the cheapest of several goals, such as the nearest of three charging docks by route or the first reachable meeting point with a moving person.
Virtual goal (imaginary goal) (G-2586)	An extra vertex added to a graph and joined to each real goal by an edge of 0 (or of that goal's extra cost); the shortest route to it ends at the best real goal, which turns a search for several goals into a search for one.
Weighted A* (G-2587)	A* that sorts by $f = g + \varepsilon h$ with a weight ε above 1, counting the guess more than the cost so far; it expands fewer states and returns a route at most ε times the shortest.
Anytime search (G-2589)	A search that can return a route whenever it is interrupted and keeps improving it while it has time, such as weighted A* run with a falling weight; it lets a robot start moving before the best route is known.
ARA* (anytime repairing A*) (G-2590)	An anytime search that runs weighted A* with a falling weight and keeps the cost-so-far values of the last run, re-expanding only states whose cost has dropped; it reaches the same routes with less repeated work (69 cells against 103 on the long trip).
Forced neighbour (G-2592)	A neighbour that the cell before could normally reach by an equally short route of its own, but which an obstacle beside the line of travel leaves reachable by a shortest route only by turning at the current cell; it makes that cell a jump point.
Jump point (G-2593)	A cell where a scan of jump point search stops because it is the goal or has a forced neighbour, a place where a route may have to turn; only jump points go on the open list.
Jump point search (JPS) (G-2594)	A* for grids with equal move costs that scans along straight lines and diagonals and puts only jump points on the open list; it skips equal routes that differ only in the order of their moves, and returns the same shortest route with far fewer open-list operations.

Term	Meaning
Any-angle search (G-2596)	Search on a grid whose routes may run at any angle between cell centres instead of only along grid moves at 0° and 45°, so they are shorter and need fewer turns.
Theta* (G-2597)	An any-angle search: A* in which a neighbour takes the expanded cell's own parent as its parent whenever the straight segment between their centres crosses no blocked cell; it gives routes of straight segments at any angle (3.193 m against 3.414 m on the room grid), but not always the shortest.
Occupancy grid (G-2340)	A grid map that stores in each cell a number between 0 and 1, such as the probability that the cell is occupied, instead of a hard 0 or 1; the standard grid map of robotics, since sensors are never sure.
Frontier (open list) (G-2547)	The vertices a search has reached but not yet expanded, waiting to be taken off one at a time; which one comes off next is what makes one search differ from another.
Euclidean distance (G-715)	The straight-line distance between two points.
Breadth-first search (BFS) (G-2550)	A search whose frontier is a queue, so it expands vertices in rings of equal moves from the start; the first route it finds has the fewest moves, which is the shortest only when every move costs the same.
Parent (back pointer) (G-2576)	The vertex from which a search reached a vertex, stored with it during the search; following parents back from the goal gives the route. In A*, a cell's parent changes whenever a shorter way in lowers its cost so far.
Priority queue (G-2568)	A collection with two operations, insert an entry with a priority and remove the entry with the best priority (for Dijkstra's algorithm, the smallest distance); it is what lets a search always pick its most promising vertex next.

Term	Meaning
Visited list (closed list) (G-2548)	The vertices a search has already expanded; a search never expands them again, so it explores each vertex once (7 expansions on the waypoint graph instead of 40 paths).
Depth-first search (DFS) (G-2552)	A search whose frontier is a stack, so it always extends the newest path and dives deep before backing up; it backs up to its last untried choice at dead ends and finds a route, but which one, and how long, depends on the order the moves are tried (22 moves where breadth-first search found 8).
Search state (G-2562)	What a search keeps for each vertex: the quantities that decide which moves are allowed and what they cost, and nothing more, such as (cell, charge left) for a robot with a battery; leaving one out breaks the Markov property and can hide routes.
Symbolic task planning (G-2600)	Choosing a sequence of actions that reaches a goal, from a description of the world in true-or-false facts such as “the cup is on the table”; it decides the order of steps (go to the table, pick up, go to the door, put down) that a route search alone cannot.